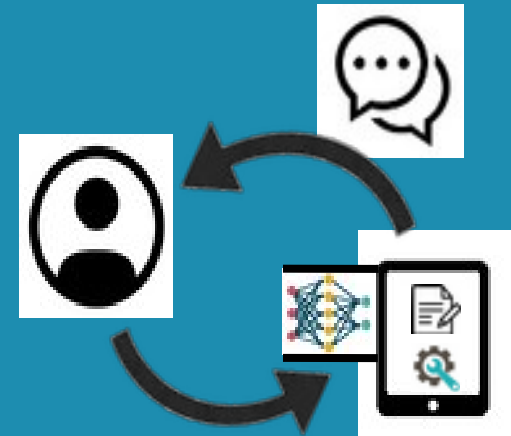
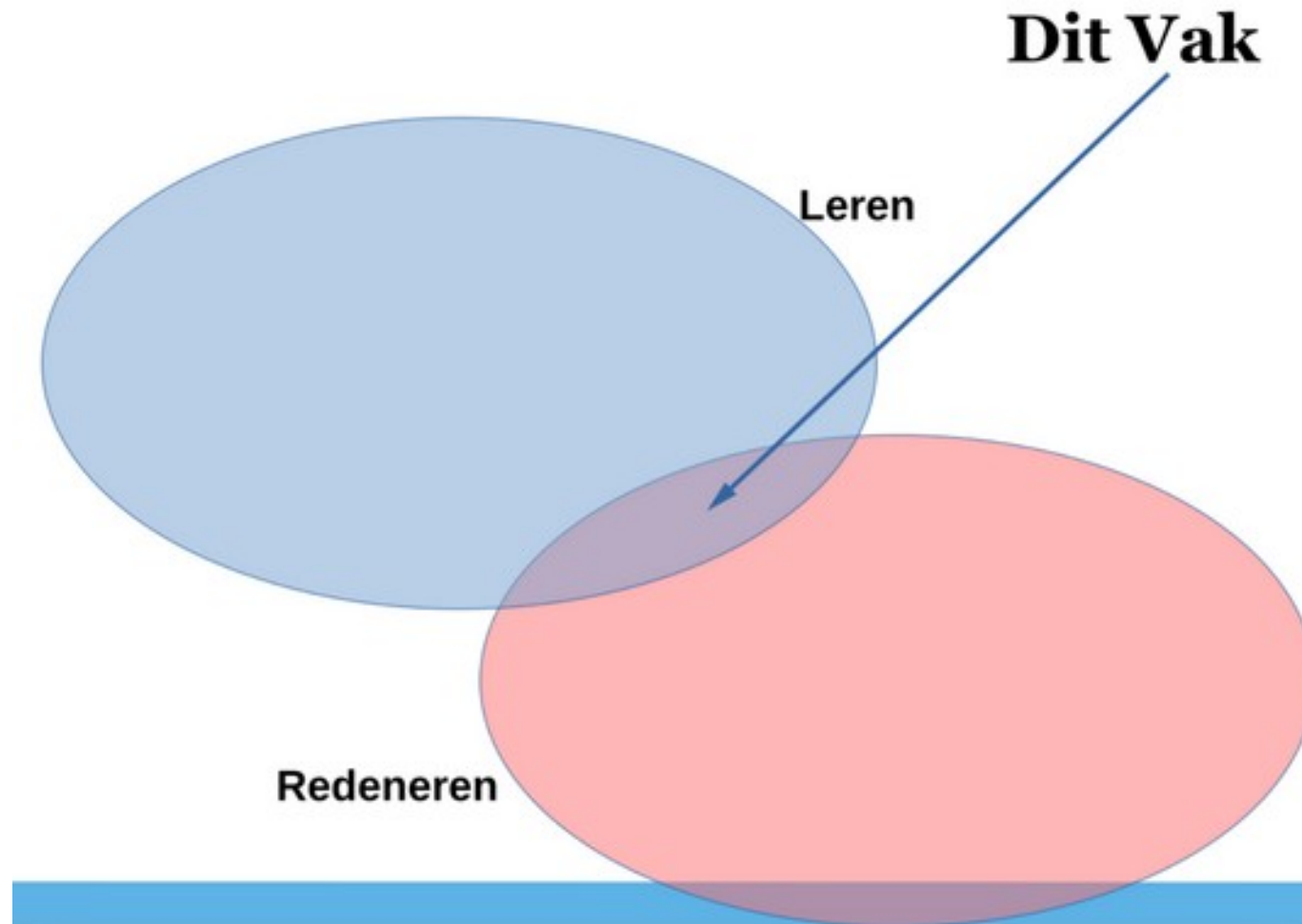


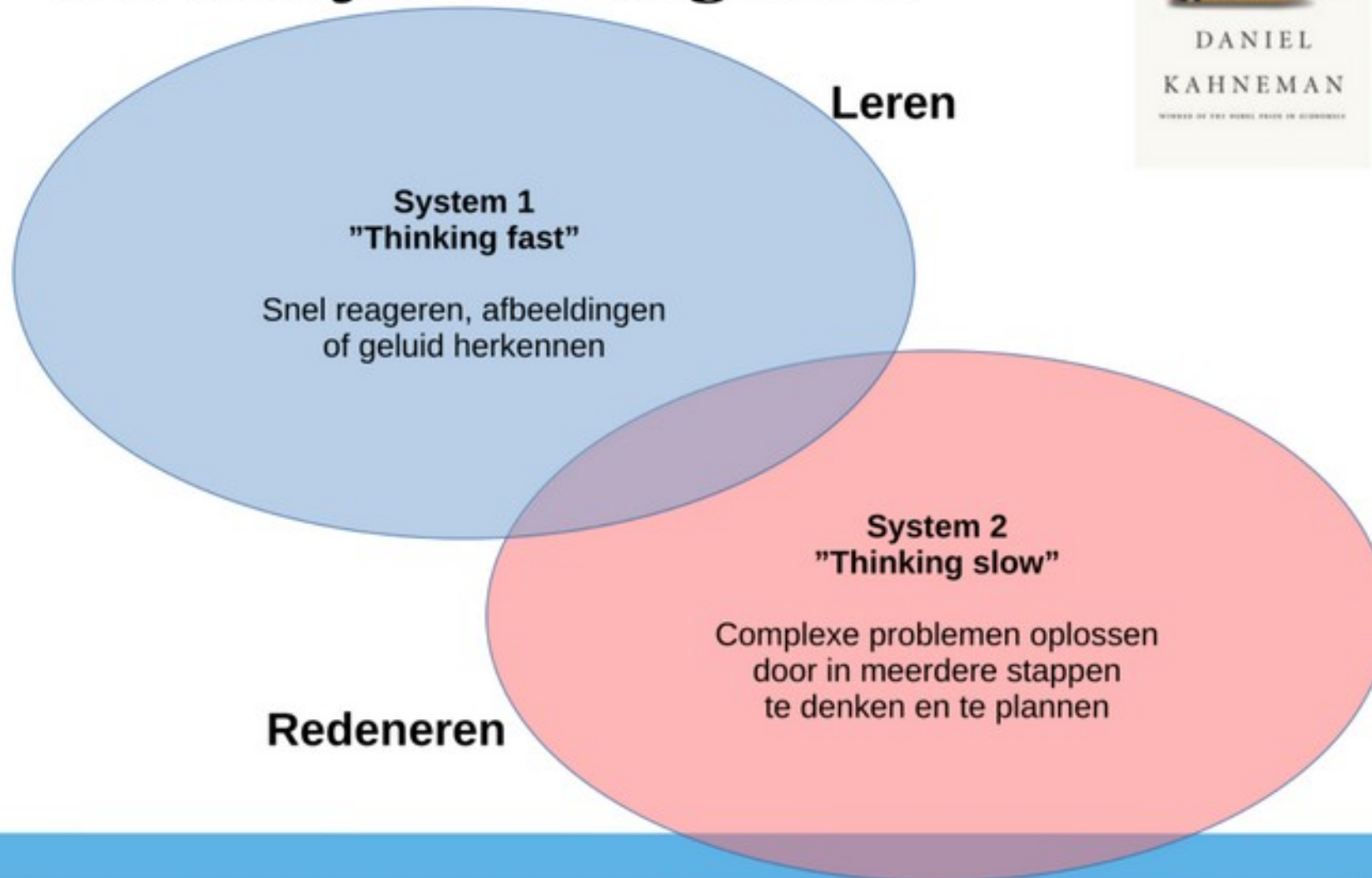
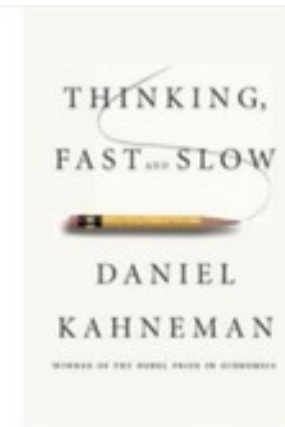
Leren en Redeneren: Optimalisatie



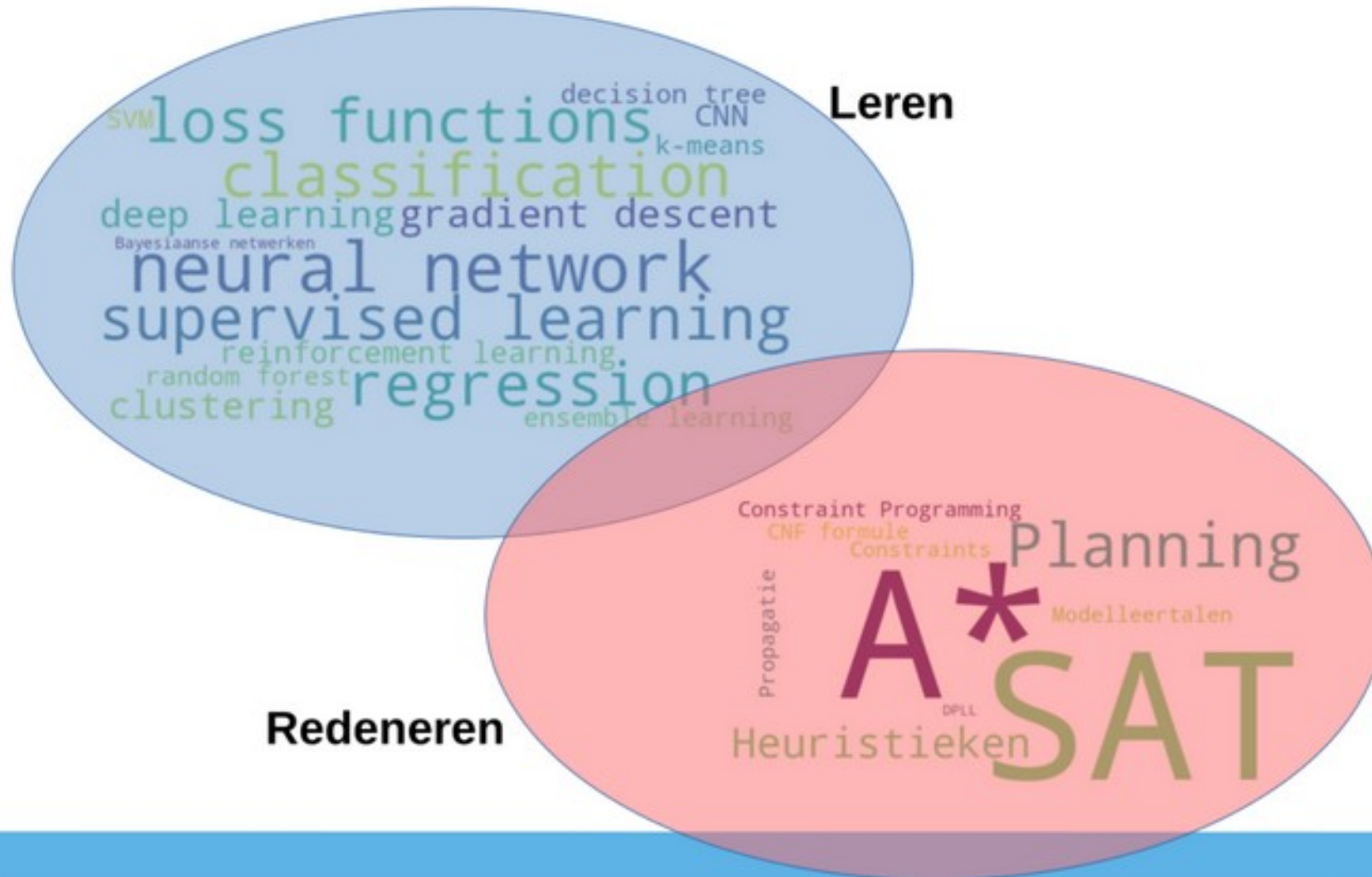
Context



Vergelijking met menselijke intelligentie

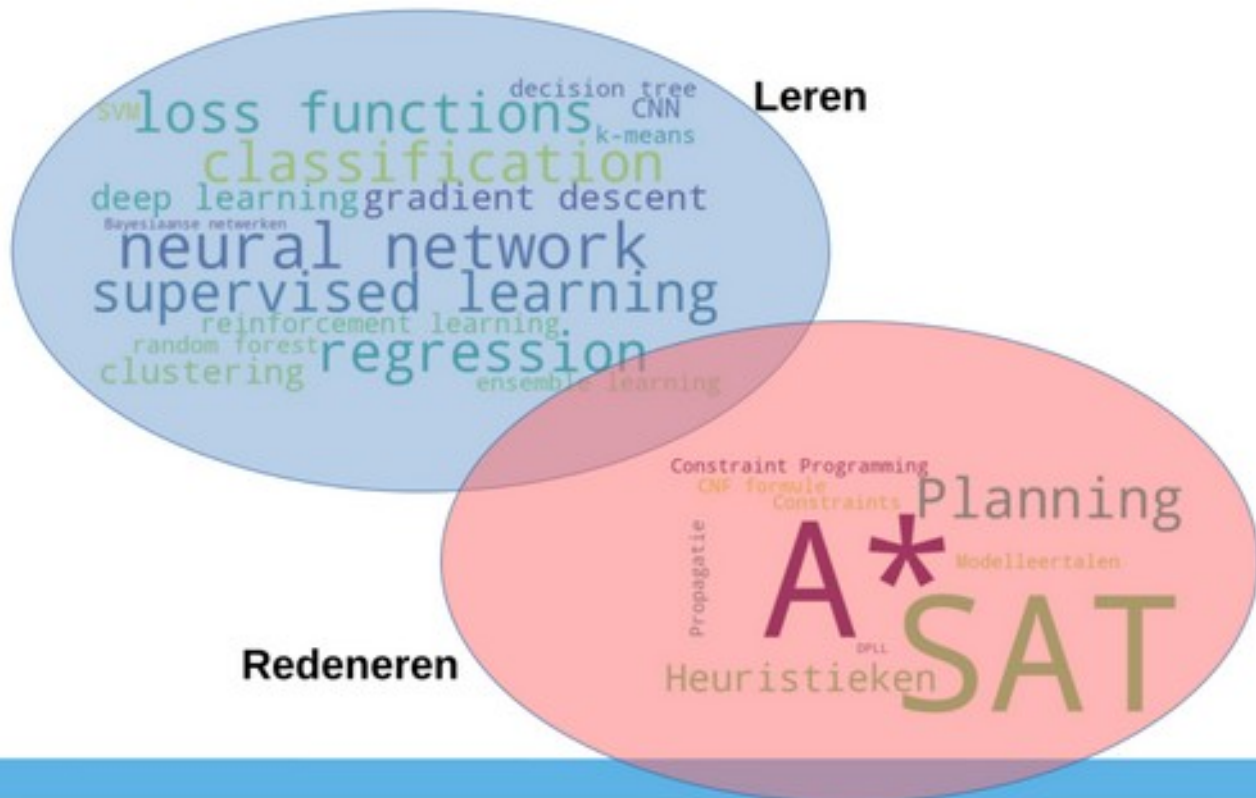


Eerdere Vakken



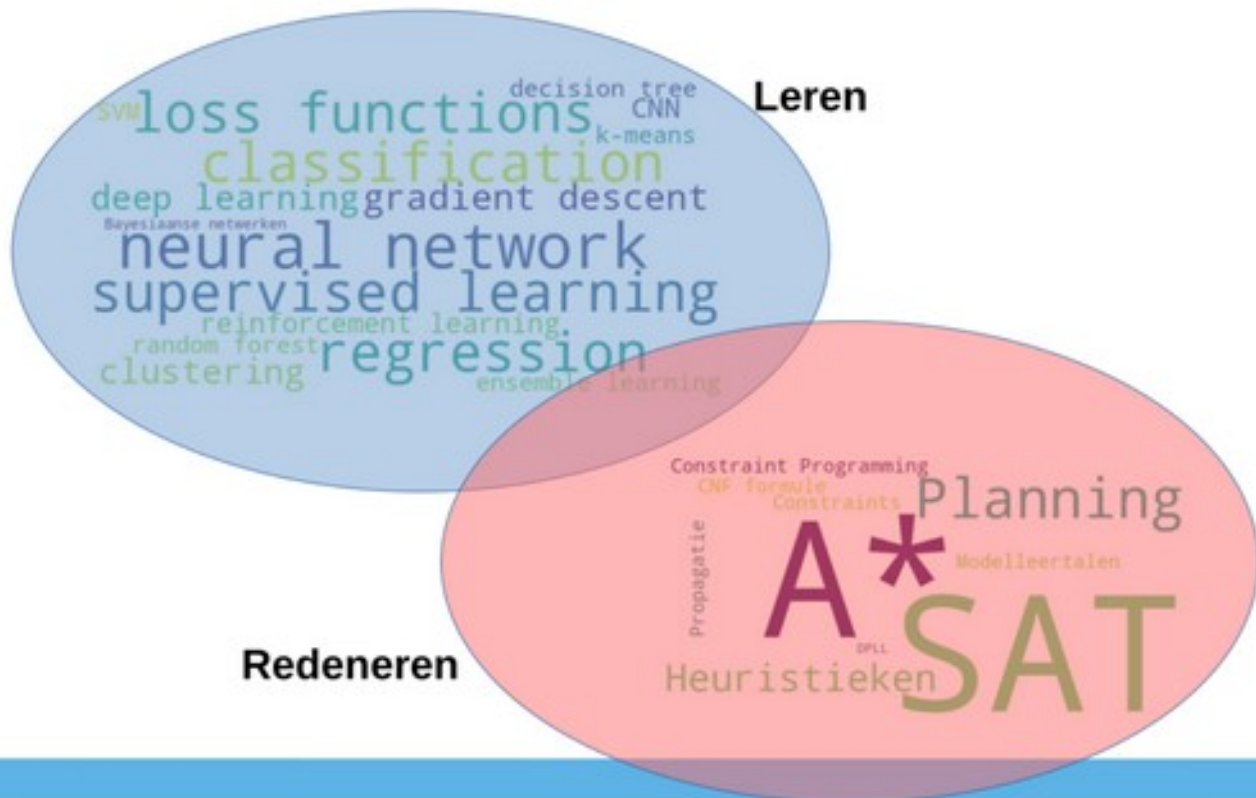
Redeneertechnieken eerdere lessen en presentaties:

- <interactief>



Redeneertechnieken deze les

- Optimalisatie



Combinatorial Optimisation

“Solving discrete, *constrained* optimisation problems”

- Vehicle Routing
- Scheduling
- Packing
- Other combinatorial problems

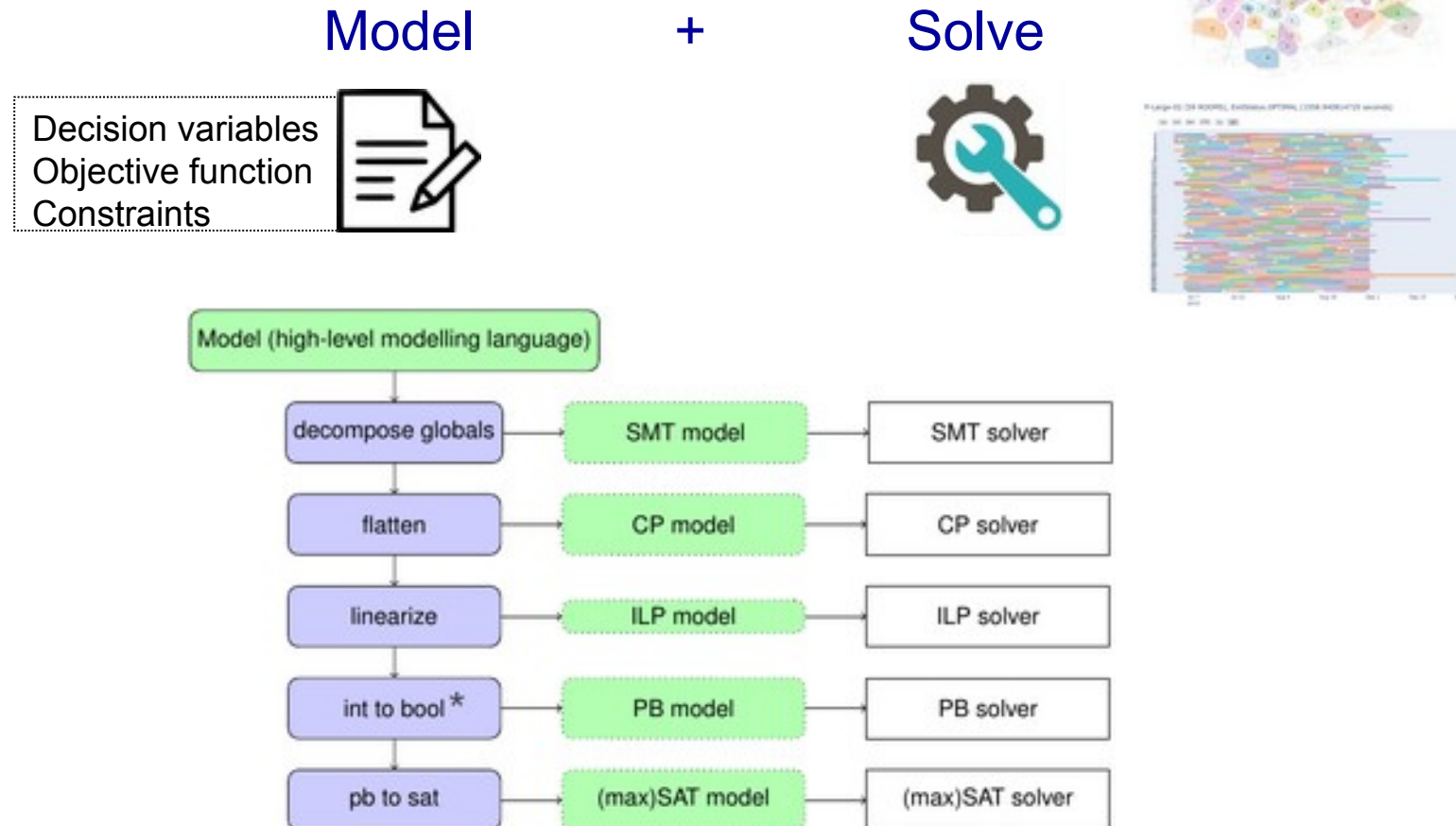


P-Large-02 (59 ROOMS), ExitStatus.OPTIMAL (1558.940814725 seconds)



[Solved and visualized with the [CPMpy](#) constraint solving library]

Optimisation: Model* + Solve paradigm



Belgian summerschool (responsible drinking) problem

Tias' Belgian beer guide

	Stella Artois, from Leuven, 5.2%, must-try factor: 5/10
	Duvel, devilish blond, 8.5%, must-try factor: 8/10
	Vedett IPA, tastefully hoppy, 6%, must-try factor: 7.5/10
	Tripel Karmeliet, strong blond, 8.4%, must-try factor: 8.2/10
	Gouden Carolus Whiskey Infused , 11.7%, must-try factor: 9.5/10
	Kriek Lindemans, sweet cherry beer, 3.5%, must-try factor: 7/10

Which beers to drink, such that you can
still pay attention tomorrow?

Model =

- Variables, with a domain
- Constraints over variables
- Optionally: an objective

- $st, du, vi, tk, gw, kl :: \{0,1\}$

- $52*st + 85*du + 60*vi + 84*tk + 117*kl + 35*gw \leq 4*52$

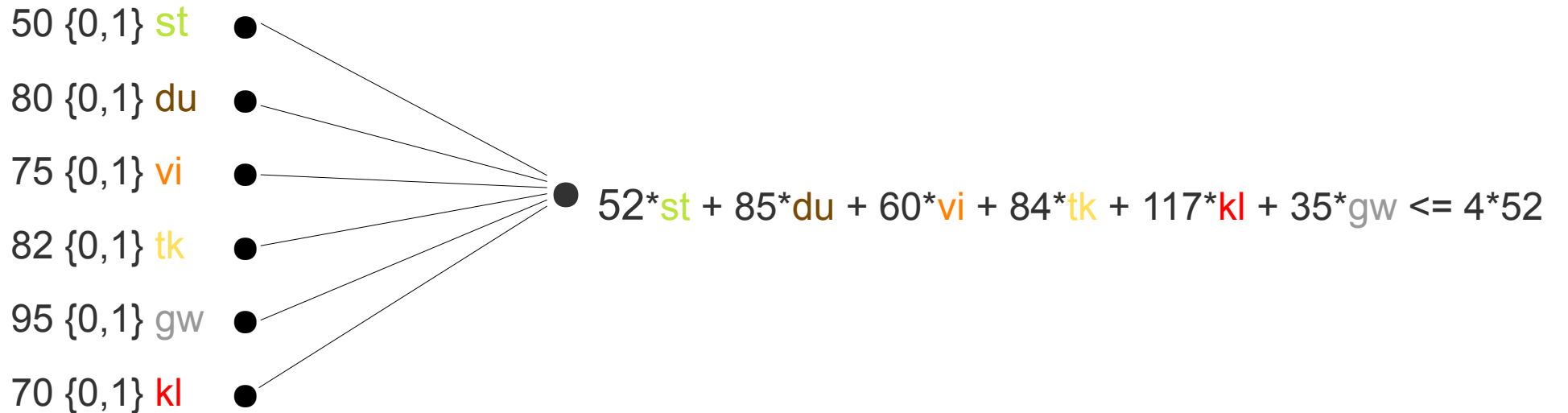
- maximize($50*st + 80*du + 75*vi + 82*tk + 95*kl + 70*gw$)



Constraint networks as factor graphs

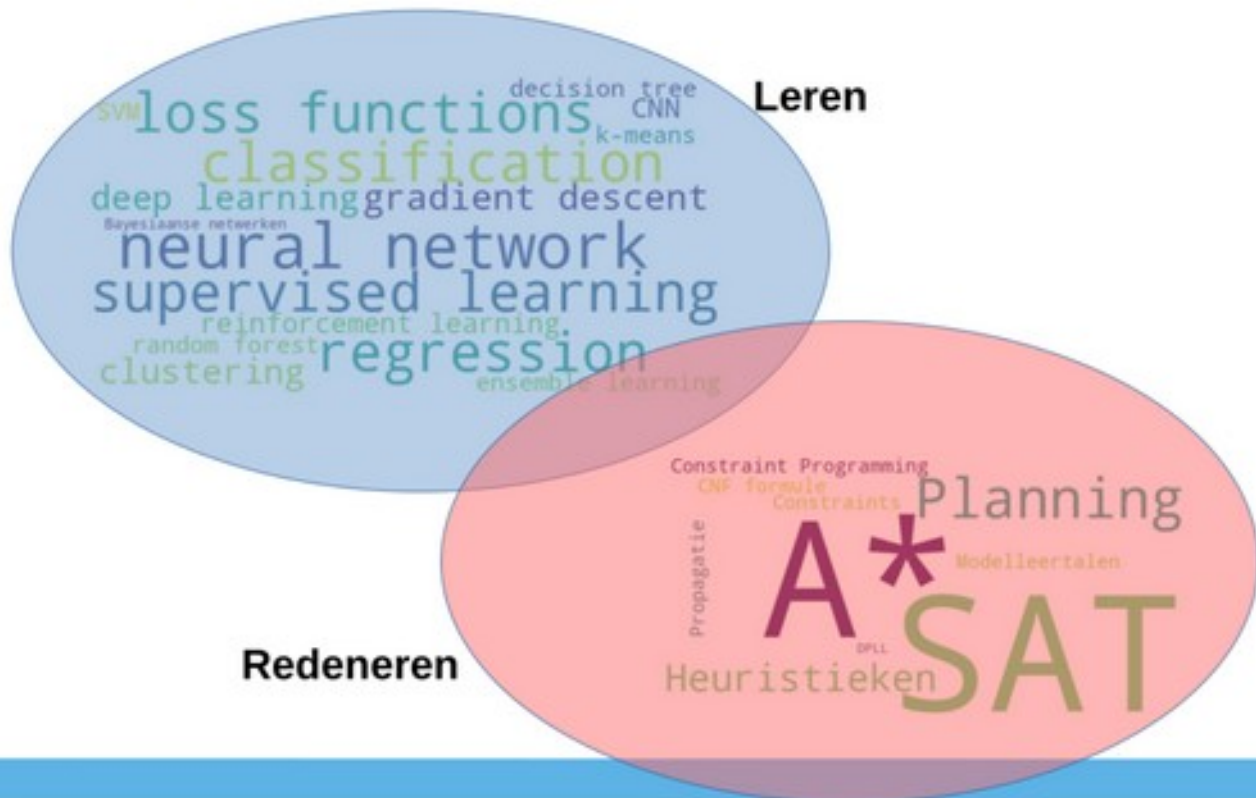
Model =

- Variables, with a domain
 - Constraints over variables
 - Optionally: an objective
- $st, du, vi, tk, gw, kl :: \{0,1\}$
 - $52*st + 85*du + 60*vi + 84*tk + 117*kl + 35*gw \leq 4*52$
 - $\text{maximize}(50*st + 80*du + 75*vi + 82*tk + 95*kl + 70*gw)$



Redeneertechnieken deze les

- Optimalisatie
(CP, MIP, MaxSAT etc)



Leren en redeneren

Henry Kautz' AAAI2020 categorisatie*:

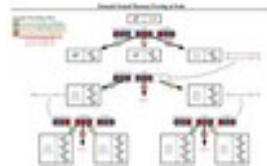
		Voorbeelden
[symbolic Neuro symbolic]	Input and output symbolic, all processing is neural	
[Neuro[Symbolic]]	Symbolic reasoning is activated by a neural engine	
[Symbolic[Neuro]]	Symbolic solver has a neural prediction component	
[Neuro + compile(Symbolic)]	Symbolic rules compiled away during training	
[Neuro → Symbolic]	Neural system provides input to symbolic system	

*Gebaseerd op bespreking in [Sarker, Zhou, Everhart, Hitzler. AI Comm 2021]

[Neuro + compile(Symbolic)]

Kennis in de *architectuur*

De **structuur** van het NN zorgt dat de uitvoer de kennis/constraints respecteert



Of: post-processing van NN uitvoer
[Neuro → Symbolic]

Kennis in de *loss* functie

De NN **parameters** worden gepenaliseerd indien de uitvoer de kennis/constraints niet respecteert (as a regularizer)

Test-time inference kan nog steeds:
[Neuro → Symbolic]

Leren en redeneren

Laatste belangrijk dimensie van leerprobleem:

- Fully supervised
(training data contains labels of all symbolic concepts)
- Weakly supervised
(training data only contains label of inference task)
[probabilistic inference: true/false,
symbolic inference: output symbol,
image generation: output image]

Example:

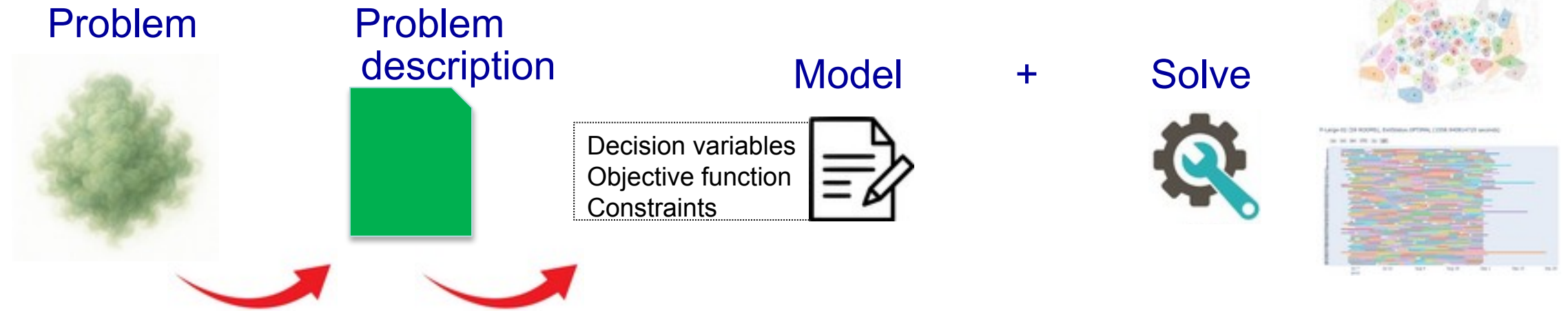


Possible NeSy learning tasks?

Lessen leren en redeneren

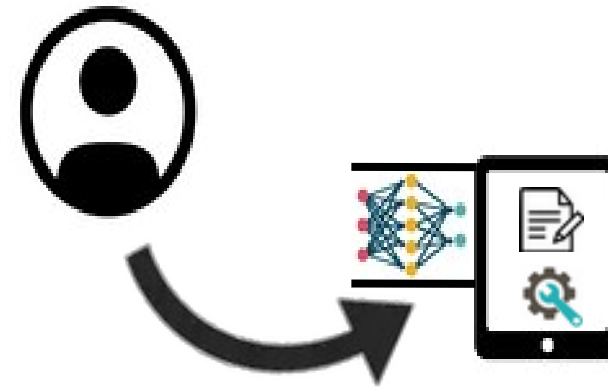
- Factor graphs
- Graph-based models
- Neuro-Symbolic AI 1
- Neuro-Symbolic AI 2
- **Optimalisatie**

Model + Solve paradigm, in practice



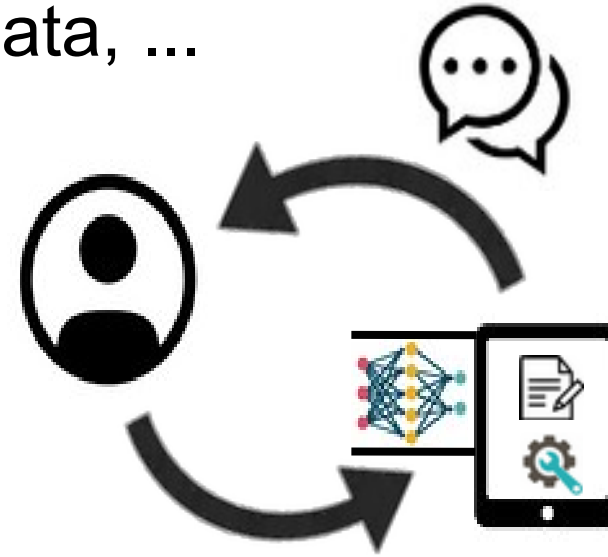
Bigger picture

- Learning from images, production data, ...
- Learning user preferences



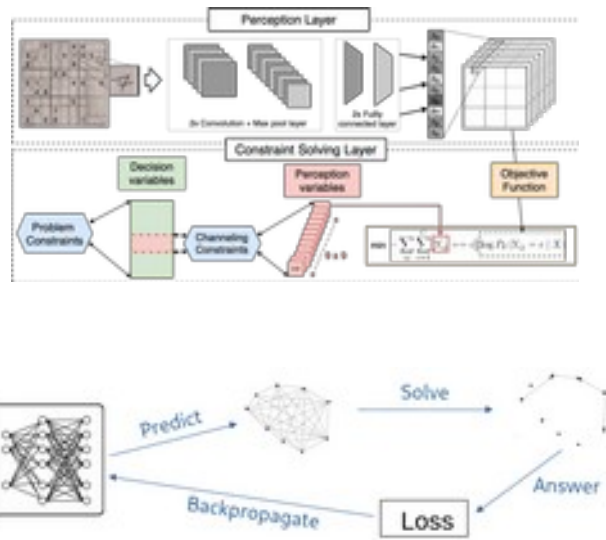
Bigger picture

- Learning from images, production data, ...
- Learning user preferences
- Stateful interaction

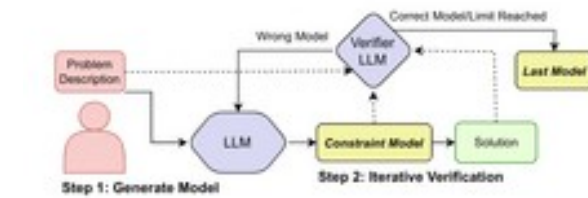


Human-Aware Technology for Optimisation

Learning



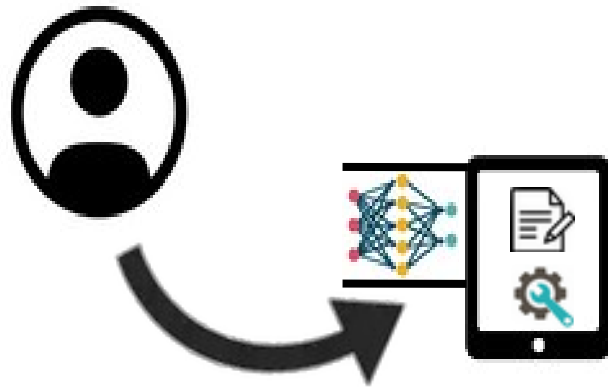
Conversational



Results Leaderboard

Run	Modelling Framework	Base LLM	Models Submitted (N)	Accuracy (N)
avgnet	CPay	Claude 4 Sonnet	100	100
sampling_10_and_self_debug_11	CPay	gpt-5-turbo-2025-08-01	100	95.34
self_debug_gpt_5	CPay	gpt-5-2025-08-01	100	95.65
sampling_10_and_self_debug_11	CPay	gpt-4.1-ultra	100	90.86

Learning: underspecified CO problems



- Underspecified parameters:
 - From (image) classifiers
 - From (contextual) regression
- Underspecified preferences:
 - Learn from historic solutions
 - Learn from interaction with user
- Underspecified structure:
 - Interactive Constraint Acquisition



Prediction + Optimisation: NeuroSymbolic

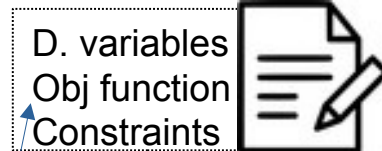
Correlated Data



+ ML +



Model



+

Solve



Part explicit knowledge:
easy to formalize

Part implicit knowledge:
hard to formalize,
easy to learn?

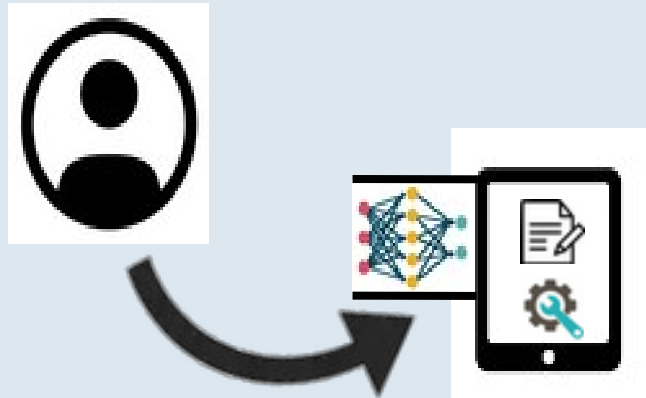


Artificial Intelligence: 2 families

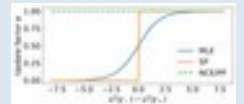
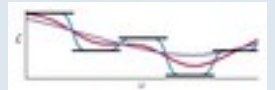
Statistical <u>L</u> earning	Formal <u>R</u> easoning
Focus on patterns and data (classifiers, neural networks)	Focus on knowledge and logical inference (planning, search)
Fast Thinking System 1: instinct, reflexes	Slow Thinking System 2: deliberate, logical, multi- step
Needs large data	Works well with small data

NeuroSymbolic AI:
integration of the two

Learning: underspecified CO problems



- Underspecified parameters:
 - **From (image) classifiers***
 - From (contextual) regression*
- Underspecified preferences:
 - Learn from historic solutions
 - Learn from interaction with user*
- Underspecified structure:
 - Interactive Constraint Acquisition



Example toy problem: Sudoku



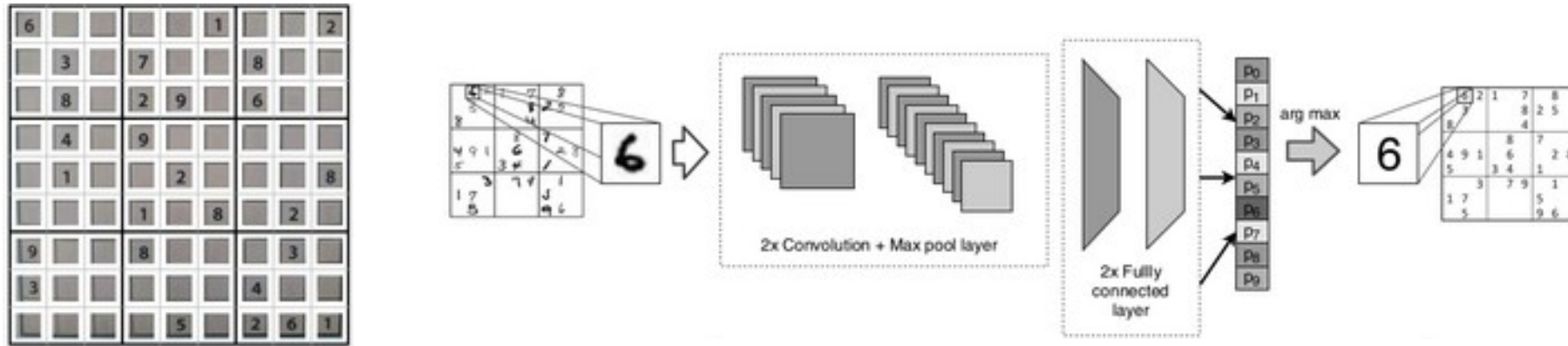
```
_ = 0 # value for empty cells
given = np.array([
    [_, _, _, 2, _, 5, _, _, _],
    [_, 9, _, _, _, _, 7, 3, _],
    [_, _, 2, _, _, 9, _, 6, _],

    [2, _, _, _, _, _, 4, _, 9],
    [_, _, _, _, 7, _, _, _, _],
    [6, _, 9, _, _, _, _, _, 1],

    [_, 8, _, 4, _, _, 1, _, _],
    [_, 6, 3, _, _, _, _, 8, _],
    [_, _, _, 6, _, 8, _, _, _]])
```



1) Recognizing the Sudoku digits



- Cut into 81 pieces (introduces additional noise)
- Predict 1-9 or empty (printed and handwritten, robust to borders and markings)
- Custom but standard ML

2) solving the sudoku

Rules of Sudoku (source: sudoku.com)

- Sudoku Rule № 1: Use Numbers 1-9

Sudoku is played on a grid of 9 x 9 spaces. Within the rows and columns are 9 "squares" (made up of 3 x 3 spaces). Each row, column and square (9 spaces each) needs to be filled out with the numbers 1-9, without repeating any numbers within the row, column or square. Does it sound complicated? As you can see from the image below of an actual Sudoku grid, each Sudoku grid comes with a few spaces already filled in; the more spaces filled in, the easier the game – the more difficult Sudoku puzzles have very few spaces that are already filled in.

	7	2			4	9		
3		4		8	9	1		
8	1	9			6	2	5	4
7		1					9	5
9					2		7	
			8		7		1	2
4		5			1	6	2	
2	3	7				5		1
				2	5	7		

Model

+

Solve

Decision variables
Constraints
Objective function



2) solving the sudoku

Decision variables
Constraints
Objective function



```
e = 0 # value for empty cells
given = np.array([
    [e, e, 2, 4, 1, e, e, e, 5],
    [1, e, 4, 3, e, e, e, e, e],
    [e, 8, e, 2, 7, 5, 3, 4, 1],

    [e, e, e, e, 3, 1, e, e, e],
    [7, 9, e, e, e, e, e, 2, e],
    [e, e, e, e, e, e, e, e, e],

    [e, e, e, e, e, 4, e, 6, e],
    [5, e, e, 8, e, e, 4, e, 9],
    [e, 4, e, 1, e, 3, 5, 7, e]])
```

```
model = Model()

# Variables
puzzle = intvar(1, 9, shape=given.shape, name="puzzle")

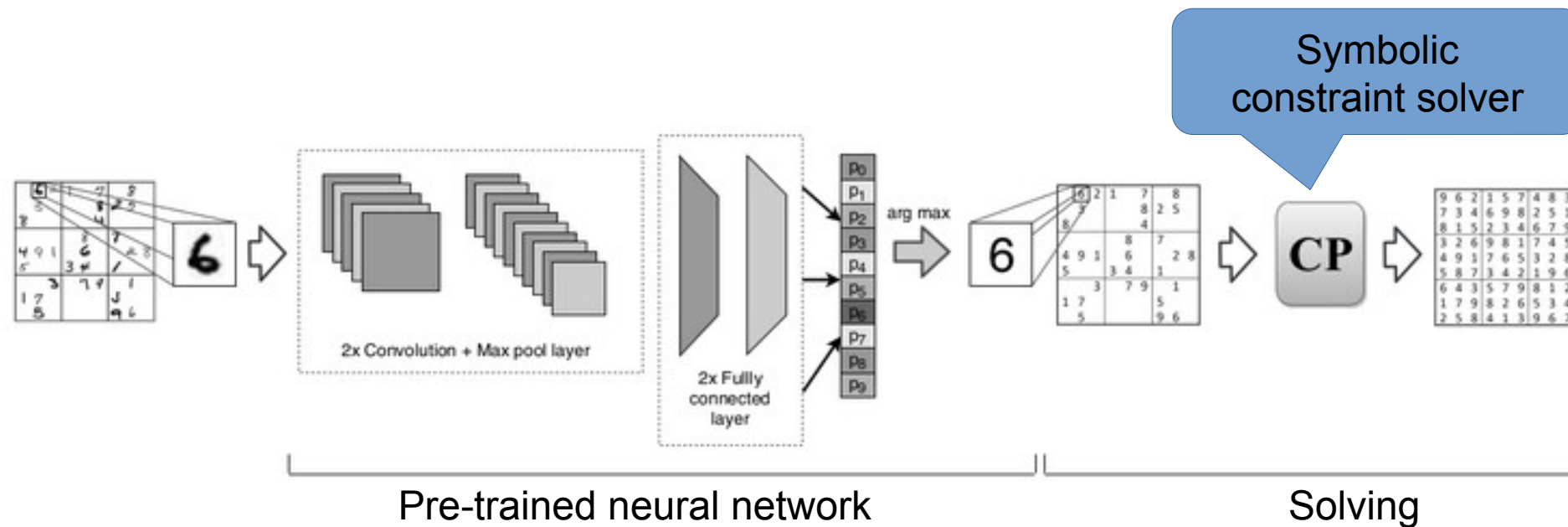
# Constraints on rows and columns
model += [AllDifferent(row) for row in puzzle]
model += [AllDifferent(col) for col in puzzle.T]

# Constraints on blocks
for i in range(0, 9, 3):
    for j in range(0, 9, 3):
        model += AllDifferent(puzzle[i:i+3, j:j+3])

# Constraints on values (cells that are not empty)
model += (puzzle[given!=e] == given[given!=e])

model.solve()
```

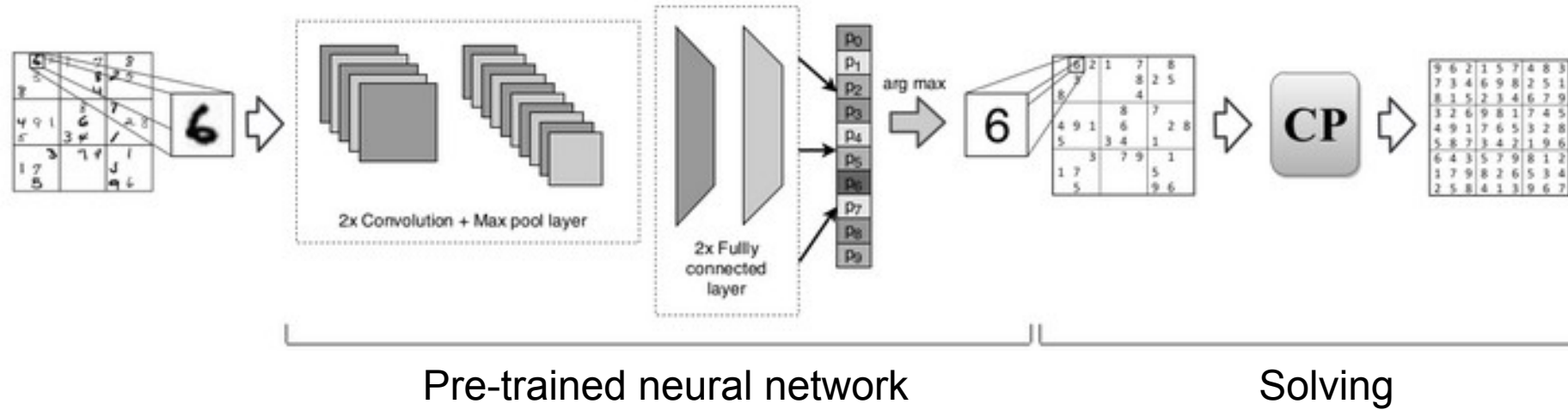
Perception + sudoku: naive baseline



Joint inference	Architecture	Grid accuracy (reasoning)	Cell accuracy (reasoning)	Unsolved	Inference time (sec.)
MNIST					
BASELINE	LeNet	73.00%±0.00%	99.56%±0.00%	26.33%±0.00%	0.011±0.001
Assistant App					
BASELINE	ViT	71.74%	99.41%	28.26%	0.052

[Mulamba, M., Mandi, J., Mahmutoğulları, A.İ. and Guns, T., Perception-based constraint solving for sudoku images. Constraints, 2024]

Perception-based constraint solving



What is going on?

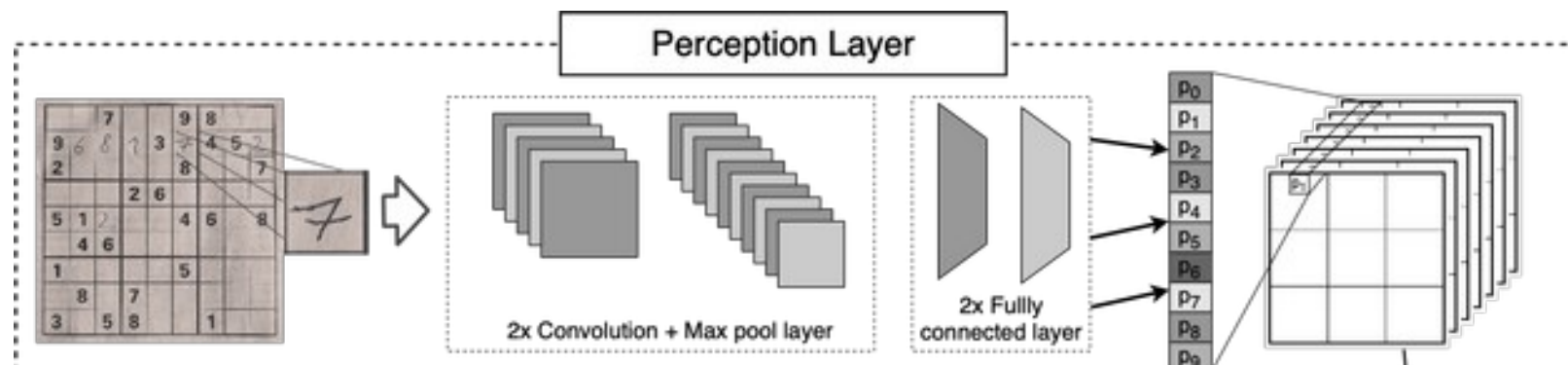
- Each cell predicts the maximum likelihood value:

$$\hat{y}_{ij} = \arg \max P(y_{ij} = k | X_{ij})$$

- But you need all 81 predictions (one for each given cell), it is a multi-output problem:
together this is the 'maximum likelihood' interpretation
- If $\text{sudoku}(\hat{y}) = \text{False}$: no solution, interpretation is wrong...

‘Visual’ sudoku: joint inference

Treat prediction as *joint inference* problem: $\hat{y} = \arg \max \prod_{ij} P(y_{ij} = k | X_{ij})$



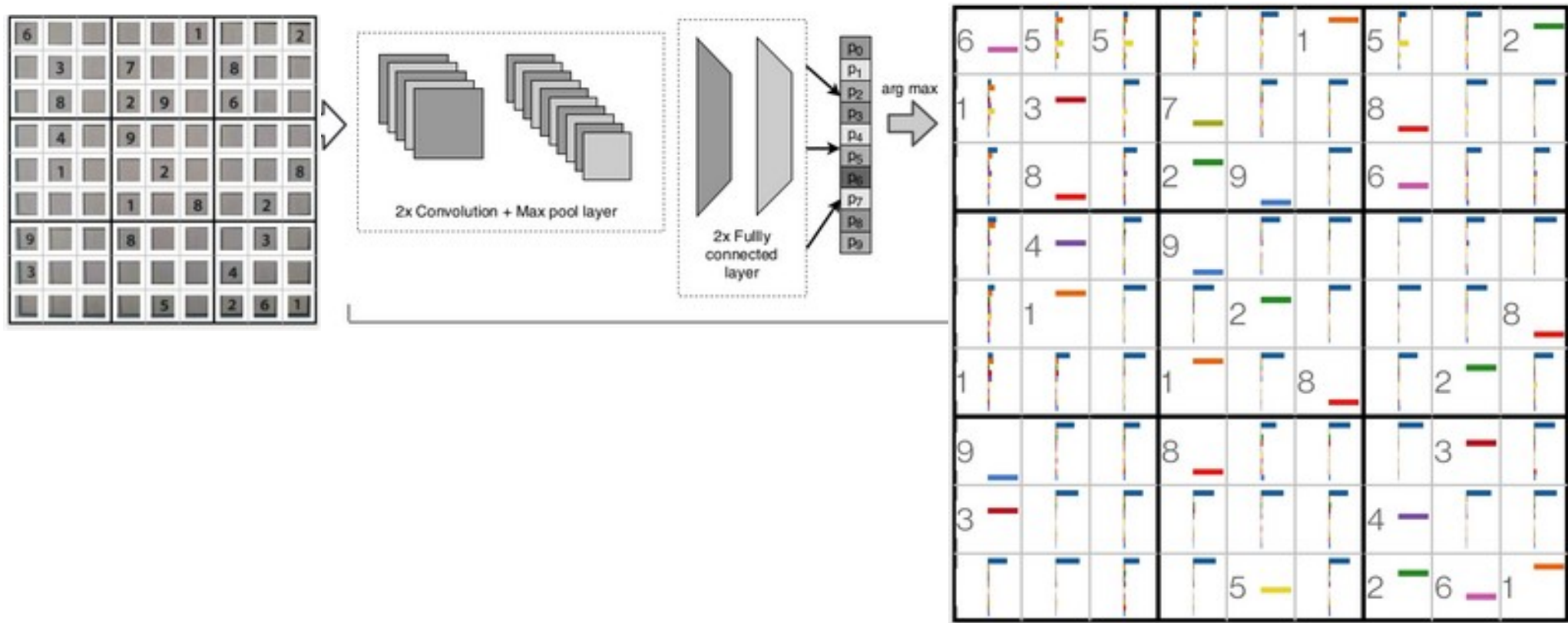
Assistant App:

Cell accuracy
- ViT: 99.41%

Grid accuracy
- ViT: 71.74%

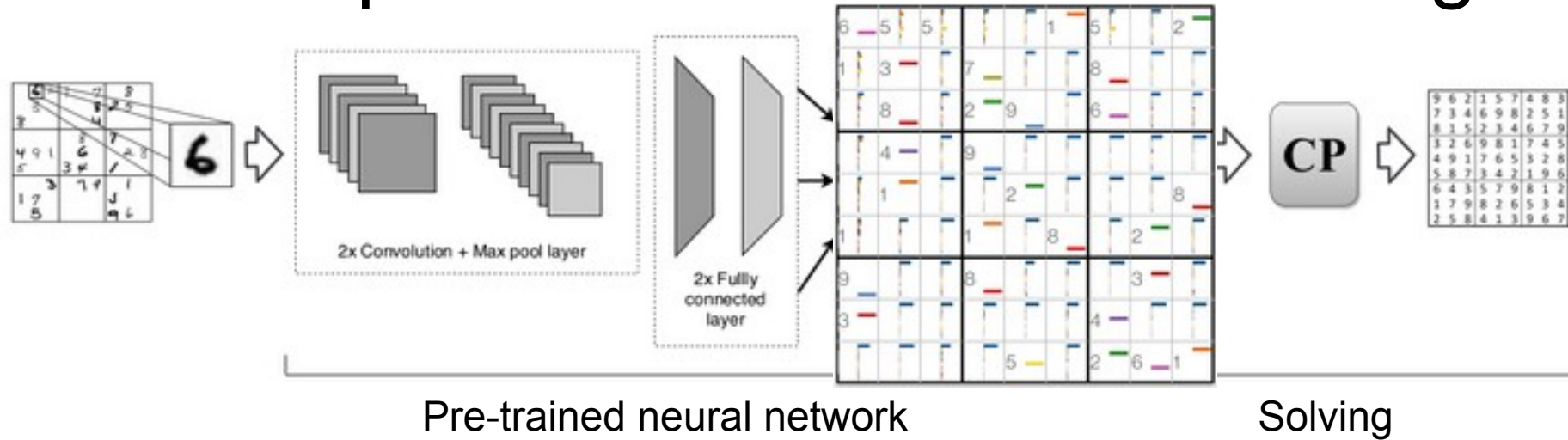
[Mulamba, M., Mandi, J., Mahmutoğulları, A.İ. and Guns, T., Perception-based constraint solving for sudoku images. Constraints, 2024]

Perception-based constraint solving



What about the *next* most likely interpretation?

Perception-based constraint solving



What about the *next* most likely interpretation?

- Treat prediction as *joint inference* problem:

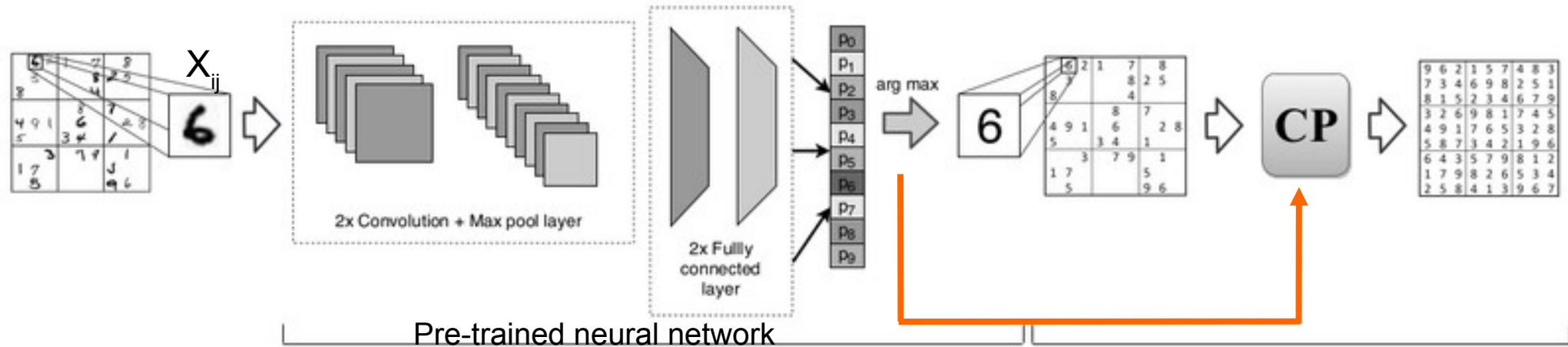
$$\hat{y} = \arg \max \prod_{ij} P(y_{ij} = k | X_{ij}) \quad \text{s.t.} \quad \text{sudoku}(\hat{y})$$

- This is the **constrained** 'maximum likelihood' interpretation

=> Structured output prediction

Used e.g. in NLP: [Punyakanok, COLING04]

Perception-based constraint solving



Can we use a constraint solver for that?

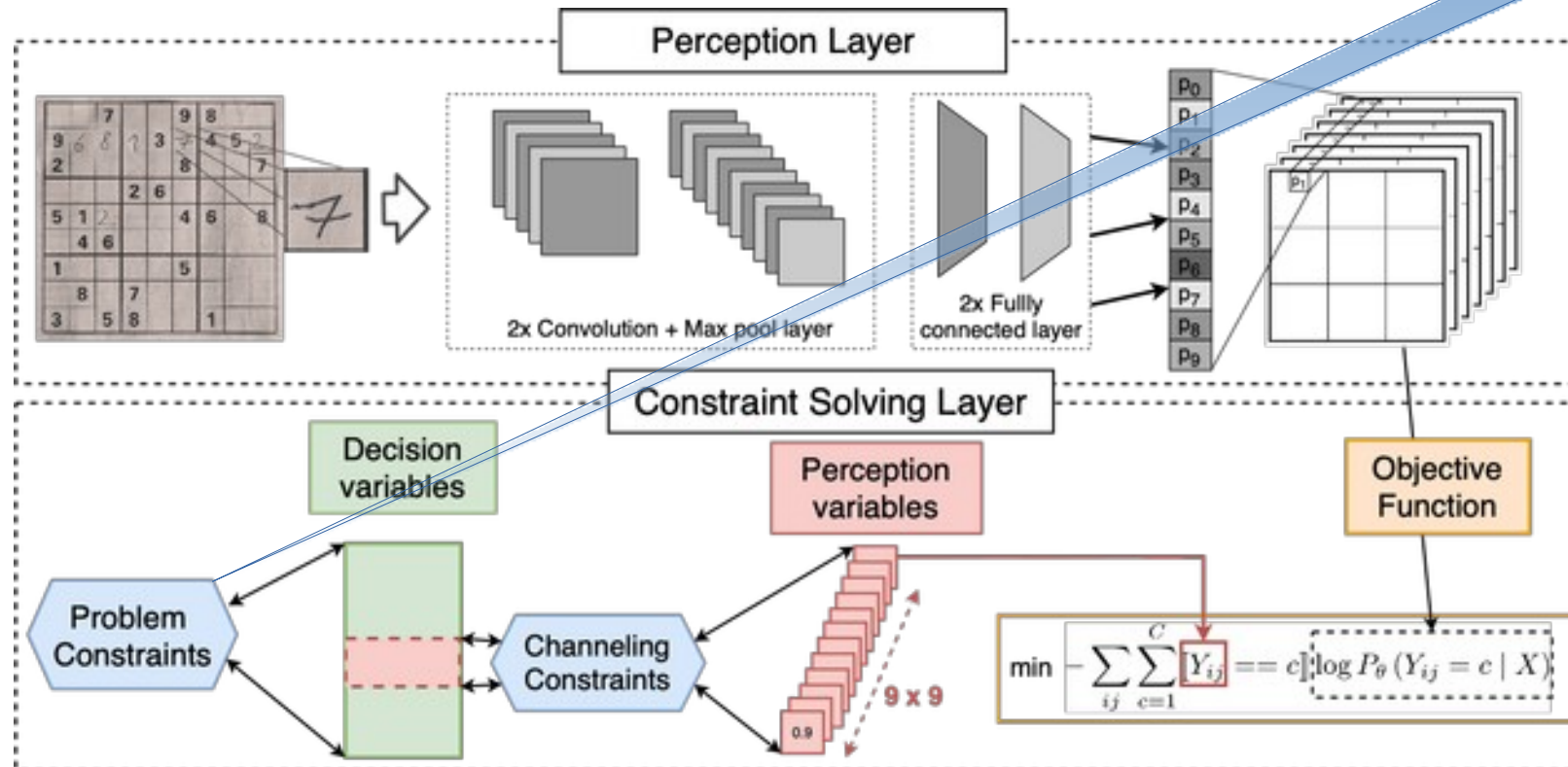
$$\hat{y} = \arg \max \prod_{ij} P(y_{ij} = k | X_{ij}) \quad \text{s.t.} \quad \text{sudoku}(\hat{y})$$

- Log-likelihood trick:

$$\min \sum_{\substack{(i,j) \in \\ \text{given}}} \sum_{k \in \{1, \dots, 9\}} \underbrace{-\log(P_{\theta}(y_{ij} = k | X_{ij}))}_{\text{constant}} * \mathbb{1}[s_{ij} = k] \quad \text{s.t.} \quad \text{sudoku}(\hat{y})$$

‘Visual’ sudoku: joint inference

Treat prediction as *joint inference* problem: $\hat{y} = \arg \max \prod_{ij} P(y_{ij} = k | X_{ij})$ s.t. $\text{sudoku}(\hat{y})$



Assistant App:

Cell accuracy
 - ViT: 99.41%
 - ViT+solve: 99.87%

Grid accuracy
 - ViT: 71.74%
 - ViT+solve: 92.39%

[Mulamba, M., Mandi, J., Mahmutoğulları, A.İ. and Guns, T., Perception-based constraint solving for sudoku images. Constraints, 2024]

Can do even better!

Are we using all available information?

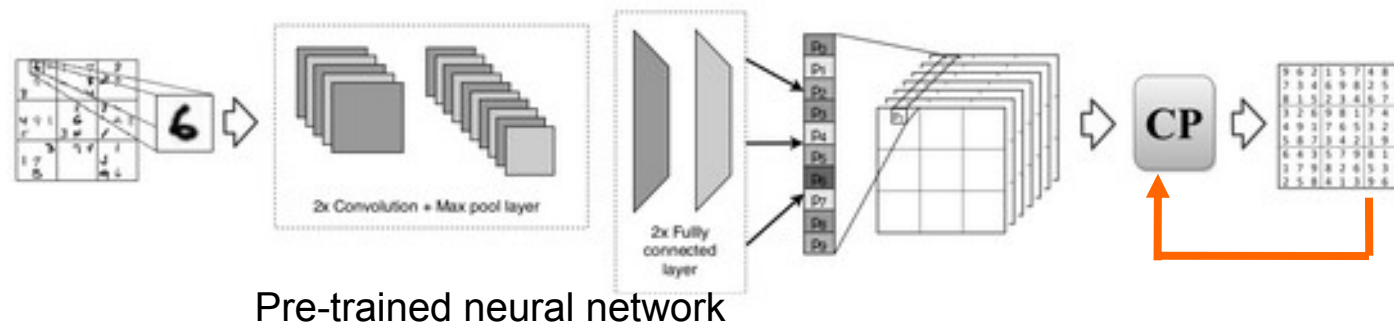
A sudoku puzzle has to have one unique solution

→ not in current constraint model: a 2nd order constraint

$$\begin{array}{ll} \underset{x}{\operatorname{argmin}} & f(X) \\ \text{subject to} & C(X) \\ & \nexists X' : X \neq X', C(X') \end{array}$$

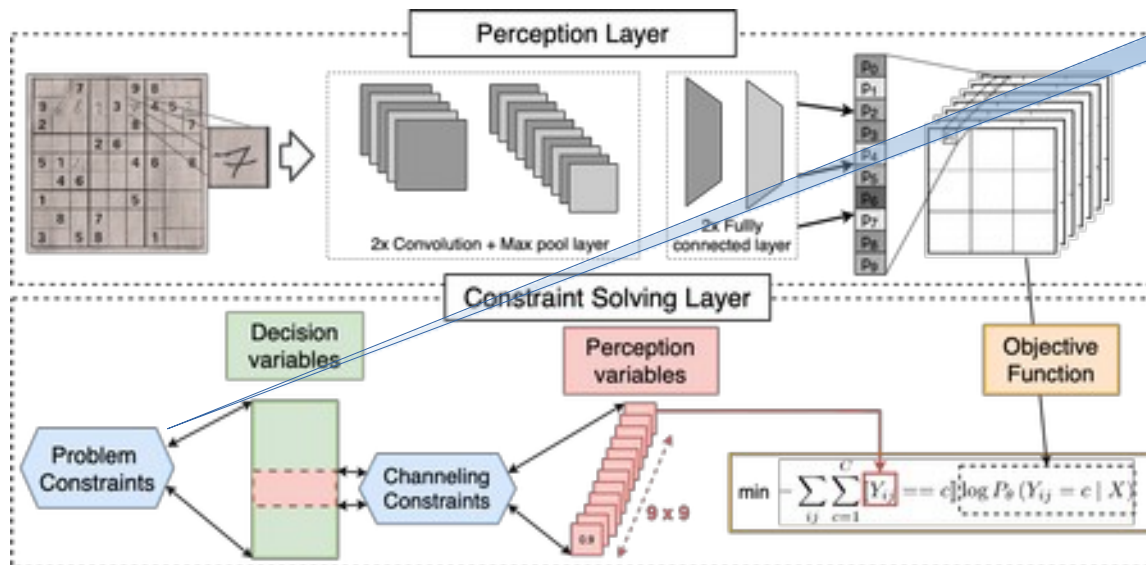
But we can add cutting planes!

if the joint max likelihood image interpretation has multiple solutions:
forbid (nogood/cutting plane) and find next most likely one!



Perception + sudoku: user errors?

Treat prediction as *joint inference* problem: $\hat{y} = \arg \max \prod_{ij} P(y_{ij} = k | X_{ij})$ s.t. $\text{sudoku}(\hat{y})$



Joint inference can correct NN mistakes

What if the user makes a mistake?
e.g. there is no valid interpretation?

Assistant App images WITH user error:

Cell accuracy (ViT): 99.05%

Grid accuracy (ViT+solve): 48.84%

→ predict printed/handwritten digits as separate classes

→ shift probability of handwritten to that of 'empty' cell

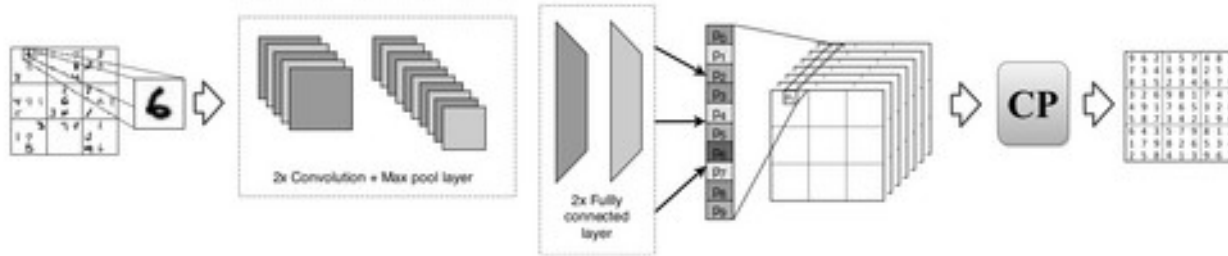
→ joint inference over **printed** interpretation

Grid accuracy (ViT+shift+solve): 81.40%

[Mulamba, M., Mandi, J., Mahmutoğulları, A.İ. and Guns, T., Perception-based constraint solving for sudoku images. Constraints, 2024]

Perception + sudoku: end-to-end learning?

- If the labels are the cell interpretations:



The reasoning *corrects* NN mistakes

- no loss/backpropagation over those
- weaker training (prelim exps: lower accuracy)

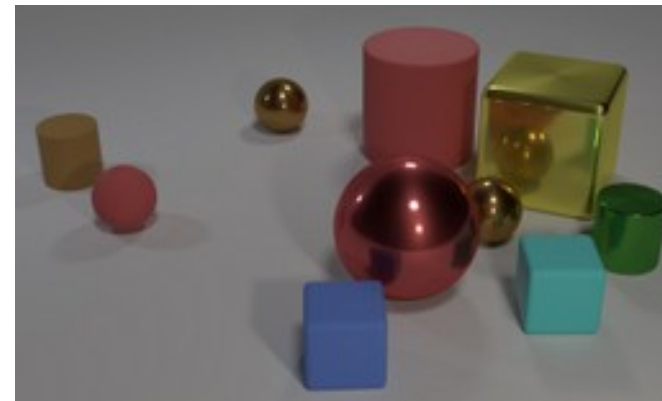
- If the data contains 'wrong' sudokus and **label is a single True/False**
 - Above does not apply, welcome SATnet, DeepProbLog, NeurASP, ...

Perception data and constraint solving

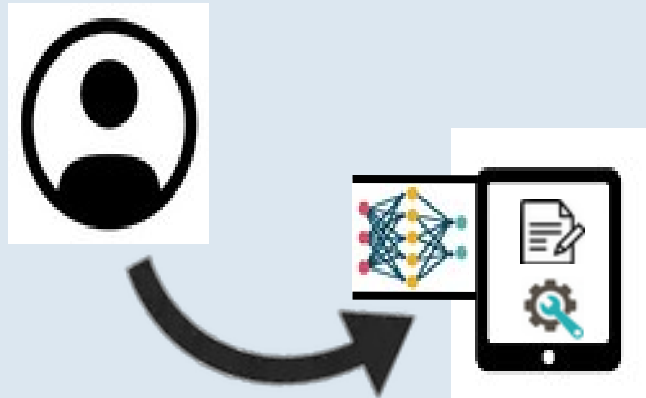
Other application settings:

- Document analysis
- Paper-based configuration problems (tax forms)
- Object-detection based reasoning
- Visual relationship detection
- ...

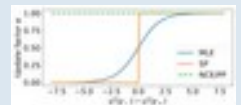
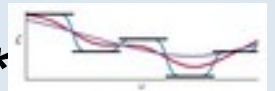
The image shows a screenshot of an Indian Individual Income Tax Return (ITR) form, specifically the 'ITR - I SAHAJ' form. The form is titled 'ITR - I SAHAJ INDIVIDUAL INCOME TAX RETURN' and includes a sub-header '(For Individuals having Income from Salaries, One House Property, Other Sources, Interest, etc.) Refer to Instructions for e-filing (2016-17)'. The assessment year is '2016-17'. The form contains various fields for personal and financial information, including name, date of birth, income, and tax details. The form is filled out with sample data, such as 'PRADIP KUMARAN KULKARNI' for the taxpayer's name and '2016-17' for the assessment year.



Learning: underspecified CO problems



- Underspecified parameters:
 - From (image) classifiers*
 - **From (contextual) regression***
- Underspecified preferences:
 - Learn from historic solutions
 - Learn from interaction with user*
- Underspecified structure:
 - Interactive Constraint Acquisition





Prediction + Optimisation: NeuroSymbolic

Correlated Data



+ ML +



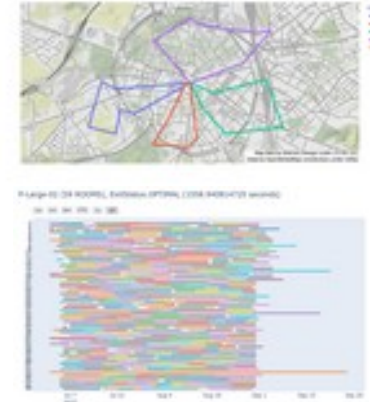
Model

D. variables
Obj function
Constraints



+

Solve



- Classification:

- ~ Images, Scans, Audio recordings, ...

- Regression:

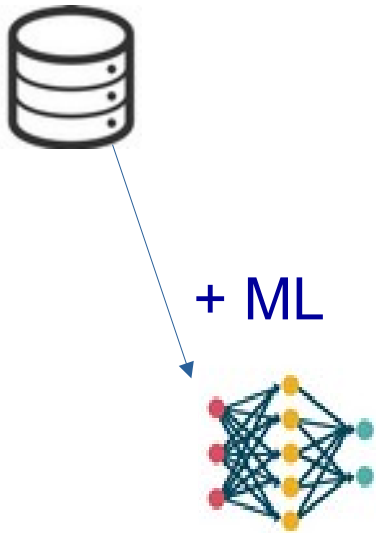
- ~ Price, Travel time, Electricity use, Value, ...

- ~ Volume, Demand, Duration, Urgency, ...



Prediction + Optimisation: ML view

Correlated Data



Standard ML

- Assumes plenty of data
- Easy to evaluate: Accuracy/MSE
- Many well-known loss functions available



Prediction + Optimisation: ML view

Correlated Data



+ ML +



Model

D. variables
Obj function
Constraints



+

Solve



ML for downstream optimisation task

- Evaluation requires solving
- What loss function to use?
- Multi-output classification, few data



Prediction + Optimisation evaluation

Correlated Data



+ ML +



Model

D. variables
Obj function
Constraints



+

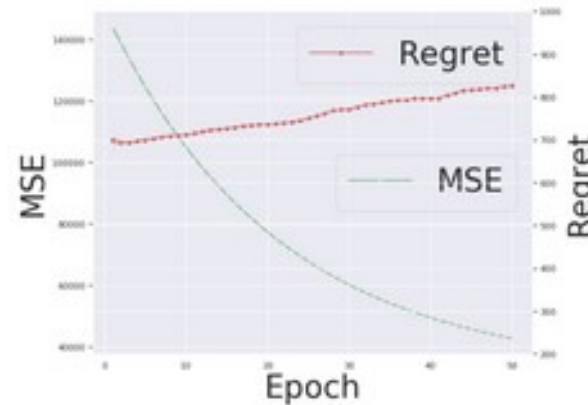
Solve



Optimal
decisions



- Downstream solution quality: regret

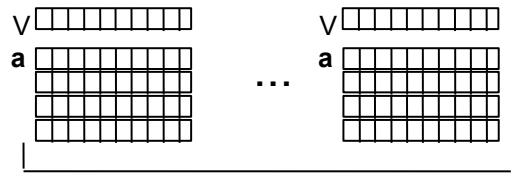


- Joint inference = joint error (regret)
- → some errors worse than others!
- → can we use it as learning signal?

[Decision-Focused Learning: Foundations, State of the Art, Benchmark and Future Opportunities: Mandi, et al. JAIR 2024]

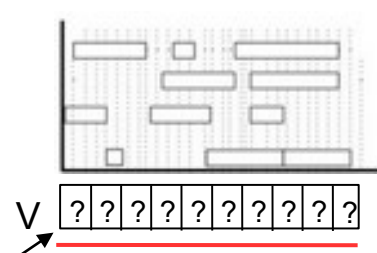
MSE loss not the best proxy for task loss....

Historic data:



learn f

apply $f(a)$

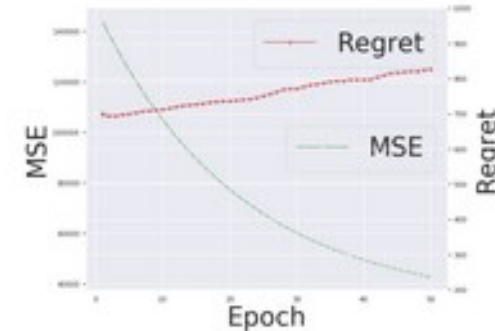


Vector of predictions

Joint inference: trades off the individual predictions

Why?

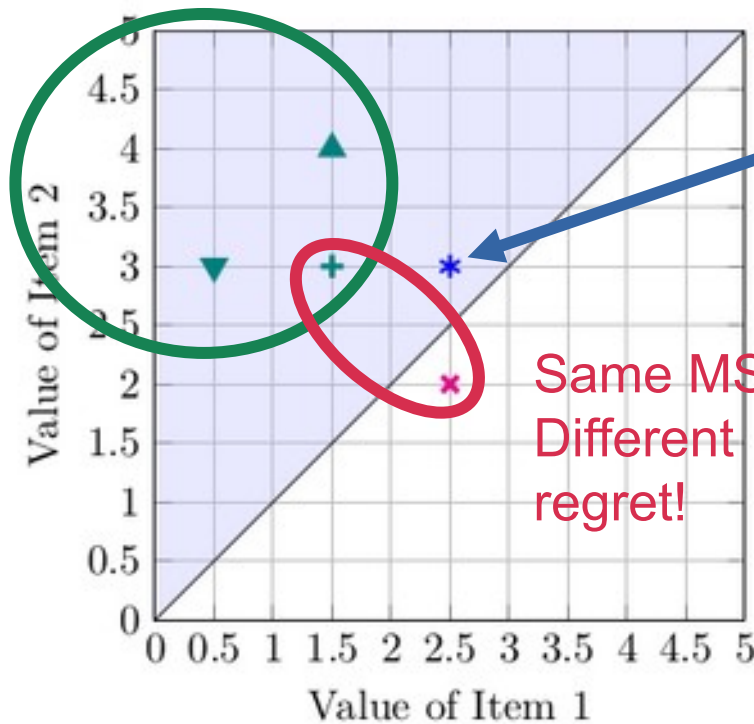
- $MSE(\hat{\mathbf{c}}, \mathbf{c}) = \frac{1}{N} \|\mathbf{c} - \hat{\mathbf{c}}\|^2$
= average of individual errors of the vector
- Joint inference = *joint* error
→ some errors worse than others!



MSE loss not the best proxy for task loss....

Example: 2-item knapsack problem, unit weights, capacity 1

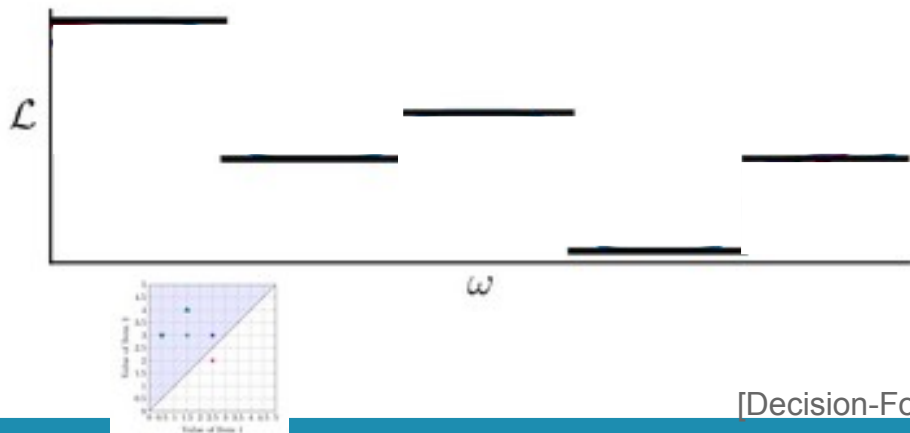
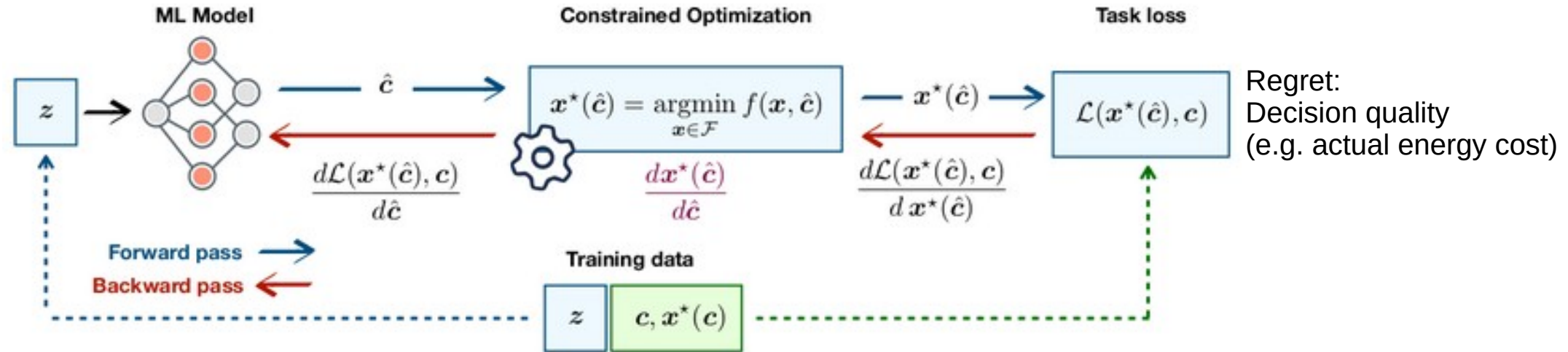
Different
MSE,
Same regret



True
values
(3, 2.5)

Same MSE,
Different
regret!

End-to-end “Decision Focused Learning”



Gradient descent? But changing the weights:

- ~ Either does not change the opt. solution (=0 gradient)
- ~ Or jumps to another optimal solution (=undefined grad)

[Decision-Focused Learning: Foundations, State of the Art, Benchmark and Future Opportunities: Mandi, et al. JAIR 2024]

Gradient descent learning for DFL

Algorithm 1 Gradient-descent in prediction-focused learning

Input: training data $D \equiv \{(\mathbf{z}_i, \mathbf{c}_i)\}_{i=1}^N$

Hyperparams: α - learning rate

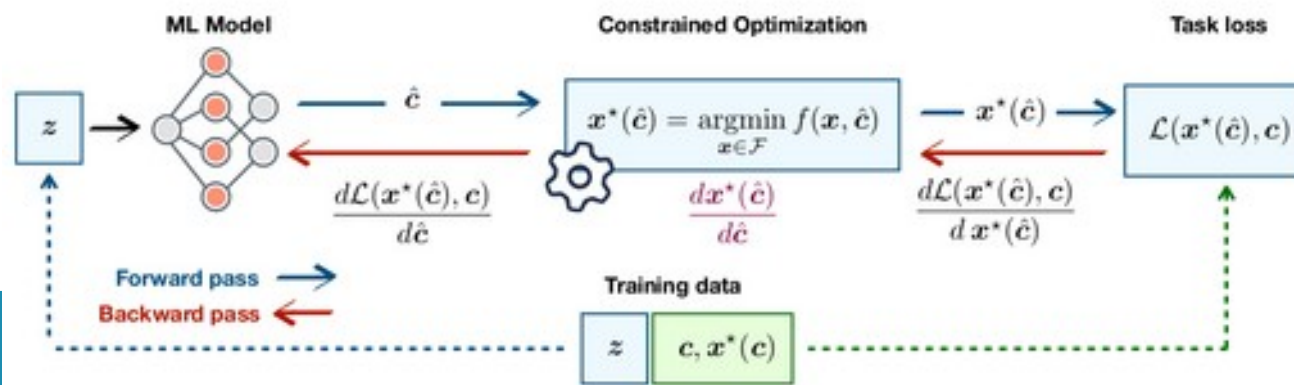
- 1: Initialize ω .
- 2: **for** each epoch **do**
- 3: **for** each instance $(\mathbf{z}, \mathbf{c})$ **do**
- 4: $\hat{\mathbf{c}} = m_{\omega}(\mathbf{z})$
- 5: $\mathcal{L} = (\hat{\mathbf{c}} - \mathbf{c})^2$
- 6: $\omega \leftarrow \omega - \alpha \frac{d\mathcal{L}}{d\hat{\mathbf{c}}} \frac{d\hat{\mathbf{c}}}{d\omega}$
- 7: **end for**
- 8: **end for**

Algorithm 2 Gradient-descent in decision-focused learning with regret as task loss

Input: $\mathcal{F}$, training data $D \equiv \{(\mathbf{z}_i, \mathbf{c}_i, \mathbf{x}^*(\mathbf{c}_i))\}_{i=1}^N$;

Hyperparams: α - learning rate

- 1: Initialize ω .
- 2: **for** each epoch **do**
- 3: **for** each instance $(\mathbf{z}, \mathbf{c}, \mathbf{x}^*(\mathbf{c}))$ **do**
- 4: $\hat{\mathbf{c}} = m_{\omega}(\mathbf{z})$
- 5: $\mathbf{x}^*(\hat{\mathbf{c}}) = \operatorname{argmin}_{\mathbf{x} \in \mathcal{F}} f(\mathbf{x}, \hat{\mathbf{c}})$
- 6: $\mathcal{L} = f(\mathbf{x}^*(\hat{\mathbf{c}}), \mathbf{c}) - f(\mathbf{x}^*(\mathbf{c}), \mathbf{c})$
- 7: $\omega \leftarrow \omega - \alpha \frac{d\mathcal{L}}{d\mathbf{x}^*(\hat{\mathbf{c}})} \frac{d\mathbf{x}^*(\hat{\mathbf{c}})}{d\hat{\mathbf{c}}} \frac{d\hat{\mathbf{c}}}{d\omega}$
- 8: **end for**
- 9: **end for**



End-to-end “Decision Focused Learning”

$$\mathbf{x}^*(\hat{\mathbf{c}}) = \underset{\mathbf{x} \in \mathcal{F}}{\operatorname{argmin}} f(\mathbf{x}, \hat{\mathbf{c}})$$

$$\frac{d\mathbf{x}^*(\hat{\mathbf{c}})}{d\hat{\mathbf{c}}}$$

Utilize

	Objective Function	Constraint Functions	Decision Variables	CO Solver
Analytical Differentiation of Optimization Mappings	Strictly Convex	Convex	Continuous	Primal-Dual Solver
Analytical Smoothing of Optimization Mappings	Linear	Linear	Continuous/ Discrete	Primal-Dual Solver
Smoothing by Random Perturbations	Linear in the Predicted Parameter	Not limited to specific form	Continuous/ Discrete	Solver agnostic
Differentiation of Surrogate Loss Functions	Linear in the Predicted Parameter	Not limited to specific form	Continuous/ Discrete	Solver agnostic

[Decision-Focused Learning: Foundations, State of the Art, Benchmark and Future Opportunities: Mandi, et al. JAIR 2024]

	Objective Function	Constraint Functions	Decision Variables	KKT Status
Analytical Differentiation of Optimalities Mapping	Scalar	Scalar	Continuous	Primal Dual Satisfy
Analytical Recombining of Optimalities Mapping	Scalar	Scalar	Continuous/Discrete	Primal Dual Satisfy
Recombining by Boolean Parametrization	Scalar in the Parametric Form	Not needed to specify form	Continuous/Discrete	Not applicable
Differentiability of Parametric State Transitions	Scalar in the Parametric Form	Not needed to specify form	Continuous/Discrete	Not applicable

1. Analytical Diff. Of Opt. Mappings

Continuous, unconstrained: $V(\mathbf{c}) = \operatorname{argmin}_{\mathbf{x}} f(\mathbf{c}, \mathbf{x})$

~ can take first-order optimality conditions $\frac{d}{d\mathbf{x}} f(\mathbf{c}, V(\mathbf{c})) = 0$
and do implicit differentiation thereof:

$$\frac{dV(\mathbf{c})}{d\mathbf{c}} = -\frac{f_{\mathbf{c}\mathbf{x}}(\mathbf{c}, V(\mathbf{c}))}{f_{\mathbf{x}\mathbf{x}}(\mathbf{c}, V(\mathbf{c}))} \quad (\text{twice differentiable } f!)$$

[Gould et al, 2016]

Continuous, constrained:

$$\mathbf{x}^* = \operatorname{argmax}_{\mathbf{x}} f(\mathbf{x})$$

~ can take KKT optimality conditions
and do implicit differentiation thereof...

$$\begin{aligned} \text{s.t. } g(\mathbf{x}) &\leq 0 \\ h(\mathbf{x}) &= 0 \end{aligned}$$

[Amos and Kolter, 2017]

	Objective Function	Constraint Functions	Decision Variables	CG-Status
✓	Linear	Linear	Continuous	Primal Dual Simplex
✓	Linear	Linear	Continuous	Primal Dual Simplex
✓	Linear in the Primal Variables	Not limited on specific form	Continuous	Interior-Point
✓	Linear in the Primal Variables	Not limited on specific form	Continuous	Interior-Point

2. Analyt. Smoothing Of Opt. Mappings

Continuous, linear $\mathbf{x}^*(\mathbf{c}) = \underset{\mathbf{x}}{\operatorname{argmax}} \mathbf{c}^\top \mathbf{x}$

~ Linear objective: 2nd derivative = 0

1) Quadratic regularisation $\mathbf{x}^*(\mathbf{c}) = \underset{\mathbf{x}}{\operatorname{argmax}} \mathbf{c}^\top \mathbf{x} - \mu \|\mathbf{x}\|_2^2$
+ impl diff of KKT
[Wilder et al, 2019]

$$= \underset{\mathbf{x}}{\operatorname{argmin}} \left\| \mathbf{x} - \frac{\mathbf{c}}{\mu} \right\|_2^2$$

2) Log-barrier regularisation
+ impl diff of KKT/HSD
[Mandi et al, 2020]

$$\mathbf{x}^*(\mathbf{c}) = \underset{\mathbf{x}}{\operatorname{argmax}} \mathbf{c}^\top \mathbf{x} + \mu \sum_i \log(x_i)$$

	Objective Function	Constraint Function	Decision Variables	ICU
Analytical Differentiation of Optimal Mappings	Binary Codes	Linear	Continuous	Fixed-Order
Analytical Sampling of Optimal Mappings	Linear	Linear	Continuous	Fixed-Order
Reconstructing by Sampling	Linear in the Parametric Space	Not Defined in specific form	Continuous	Adaptive
Differentiation of Nonlinear Sub-Problems	Linear in the Parametric Space	Not Defined in specific form	Continuous	Adaptive

2. Analyt. Smoothing Of Opt. Mappings

Discrete, linear

- ~ Relax to linear program [Wilder et al, 2019]
+ any of the above
- ~ Tightness of relaxation? [Ferber et al, 2020]
=> Tighten with cutting planes at root node of MIP

Also differentiable relaxations for MaxSAT, submodular problems etc... (see survey)

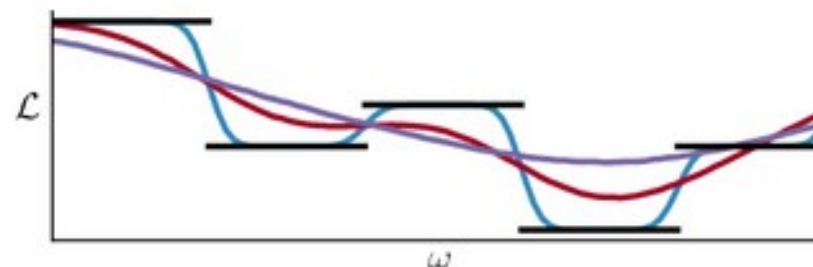
	Objective Functions	Constraint Functions	Decision Variables	CGO Status
Analytical Differentiation of Optimalities (Hager)	Binary Values	Linear	Continuous	Partial Dual Active
Analytical Differentiation of Optimalities (Hager)	Linear	Linear	Continuous/Discrete	Partial Dual Active
Smoothing by Random Perturbations	Linear in the Primal Variables	Not limited to specific form	Continuous/Discrete	Sub-optimal
Differentiability of the Primal Problem	Linear in the Primal Variables	Not limited to specific form	Continuous/Discrete	Sub-optimal

3. Smoothing by random perturbation

Perturb the cost vector by sampling from a well-chosen distribution:

$$\mathbf{x}_{\epsilon}^*(\mathbf{c}) = \mathbf{x}^*(\mathbf{c} + \epsilon\boldsymbol{\eta})$$

Stochastic smoothing (=avg over k samples):
continuously differentiable



Solver-agnostic and optimisation-as-a-layer (anywhere in NN)!

[Pogangic et al, 2020; Berthet et al, 2020, Niepert et al, 2021, ...]

	Objective Function	Constraint Function	Decision Variables	CG Subclass
Analysis of Differentiability of Optimal Value	Linear	Linear	Continuous	Primal Dual
Analysis of Differentiability of Optimal Value	Linear	Linear	Continuous	Primal Dual
Analysis of Differentiability of Optimal Value	Linear	Linear	Continuous	Primal Dual
Analysis of Differentiability of Optimal Value	Linear	Linear	Continuous	Primal Dual
Analysis of Differentiability of Optimal Value	Linear	Linear	Continuous	Primal Dual
Analysis of Differentiability of Optimal Value	Linear	Linear	Continuous	Primal Dual
Analysis of Differentiability of Optimal Value	Linear	Linear	Continuous	Primal Dual
Analysis of Differentiability of Optimal Value	Linear	Linear	Continuous	Primal Dual
Analysis of Differentiability of Optimal Value	Linear	Linear	Continuous	Primal Dual
Analysis of Differentiability of Optimal Value	Linear	Linear	Continuous	Primal Dual

4. Differentiation of surrogate losses

SPO+ loss [Elmachtoub and Grigas, 2017 2021]

Proposes a convex surrogate upper bound on regret:

$$\mathcal{L}_{SPO+}(\mathbf{x}^*(\hat{\mathbf{c}})) = 2\hat{\mathbf{c}}^\top \mathbf{x}^*(\mathbf{c}) - \mathbf{c}^\top \mathbf{x}^*(\mathbf{c}) + \max_{\mathbf{x} \in \mathcal{F}} \{\mathbf{c}^\top \mathbf{x} - 2\hat{\mathbf{c}}^\top \mathbf{x}\}$$

... and has a simple subgradient

$$\underline{\mathbf{x}^*(\mathbf{c})} - \underline{\mathbf{x}^*(2\hat{\mathbf{c}} - \mathbf{c})} \in \partial \mathcal{L}_{SPO+}$$

True optimal
solution

Optimal solution under predicted,
perturbed cost vector (perturbed by error: $\hat{\mathbf{c}} - \mathbf{c}$)

requires knowing the true cost vector $\mathbf{c}$ (*fully supervised*)

Relation to *ranking*

Loss: higher probability to v^* than to any other v^s in V ?

Key observation:

“The objective function induces a ranking over feasible solutions”

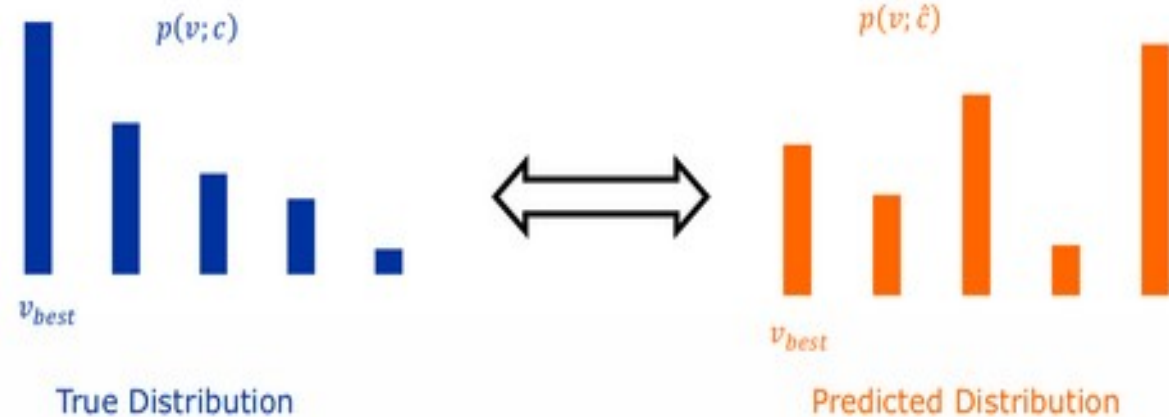
	Obj with true costs	Obj with predicted
Sol 1 [0,2,1,3,0]	12 (rank: 1)	14 (rank: 3)
Sol 2 [0,1,2,3,0]	15 (rank: 2)	10 (rank: 1)
Sol 3 [0,2,3,1,0]	16 (rank: 3)	11 (rank: 2)
Sol 4 [0,3,1,2,0]	23 (rank: 4)	16 (rank: 4)
Sol 5 [0,3,2,1,0]	28 (rank: 5)	18 (rank: 5)

[Mandi, Mulamba, Bucarey, Guns, Decision-focused learning: through the lens of learning to rank, ICML2022]

L2R for DFL: Listwise

Discrete exponential distribution
in solution space

$$p(v; c) = \begin{cases} \frac{1}{Z} \exp(-f(v, c)/\tau) & v \in V \\ 0 & v \notin V \end{cases}$$



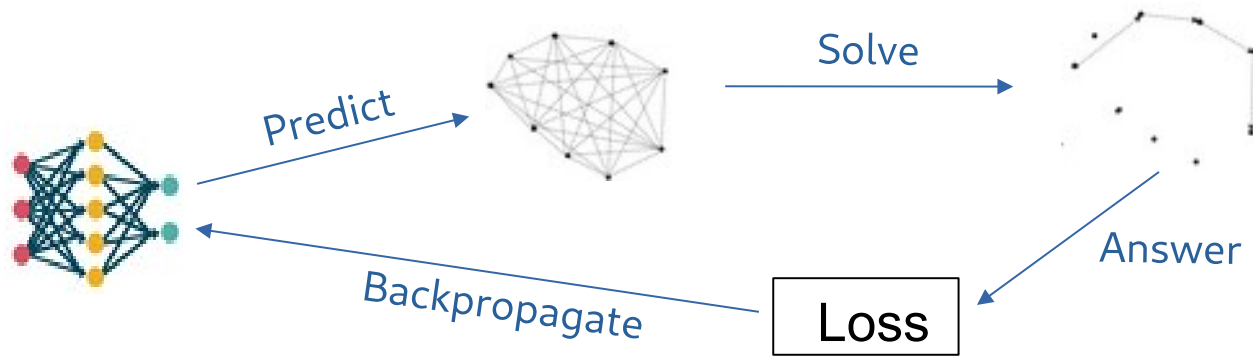
Given a finite sample of feasible solutions S , for which we compute Z' :
we obtain 2 empirical distributions (one for true costs, one for predicted)

=> Can use the standard Kullback-Leibler Divergence loss!

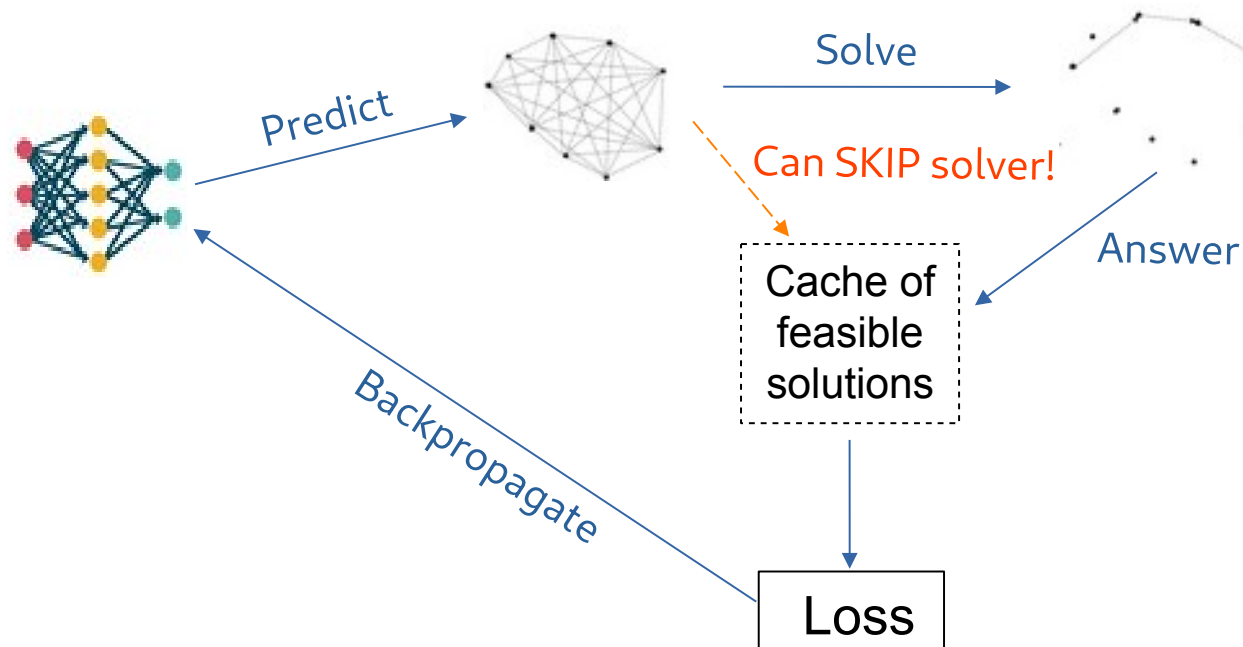
[Mandi, Mulamba, Bucarey, Guns, Decision-focused learning: through the lens of learning to rank, ICML2022]

Decision-focused learning with L2R

2nd Key bottleneck: repeatedly calling the solver



Using a solution cache

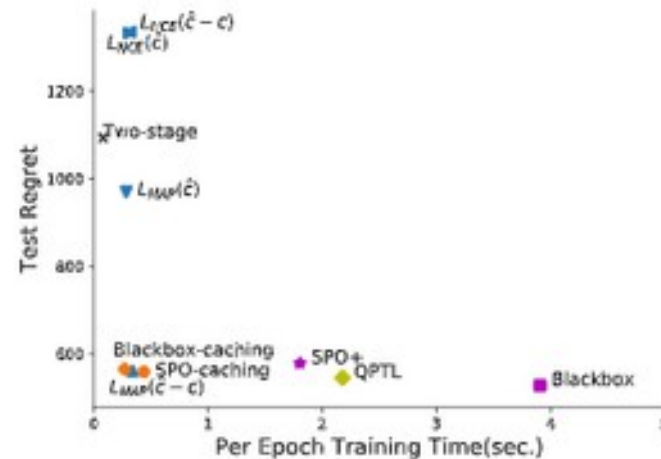
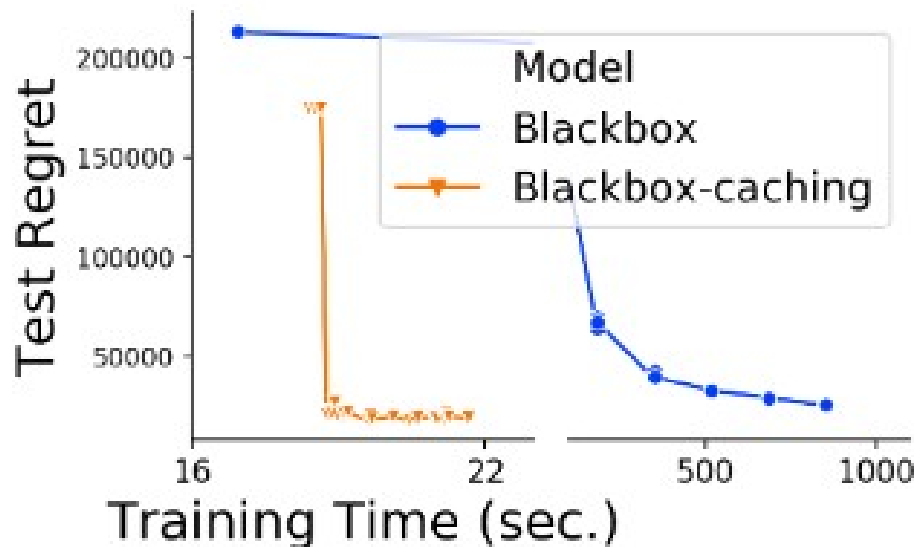


Can SKIP the solver! (e.g. only call solver 5% of time = 95% time reduction)

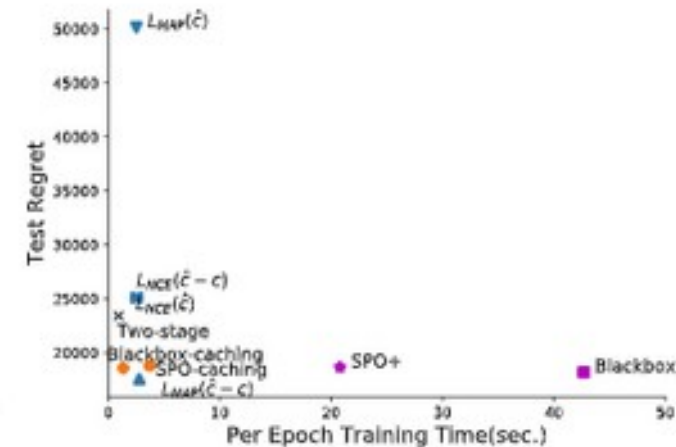
Works with ANY solver-agnostic DFL loss function... (3) and (4)

Experimental results

Caching scheme compatible with **all** methods that call a blackbox solver
(*do linear lookup in cache instead, e.g. 90% of time*)



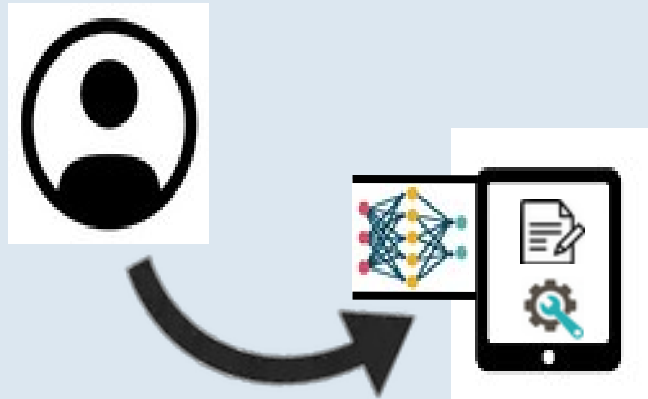
(a) Knapsack-120



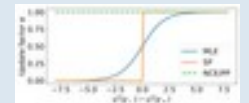
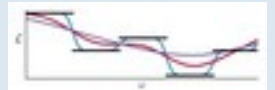
(b) Energy-3

[Mulamba, Mandi, Diligenti, Lombardi, Bucarey, Guns. Discrete solution pools and noise-contrastive estimation for predict-and-optimize IJCAI 2021]

Learning: underspecified CO problems



- Underspecified parameters:
 - From (image) classifiers*
 - From (contextual) regression*
- Underspecified preferences:
 - Learn from historic solutions
 - **Learn from interaction with user***
- Underspecified structure:
 - Interactive Constraint Acquisition



Tacit knowledge (user preferences)

Example:

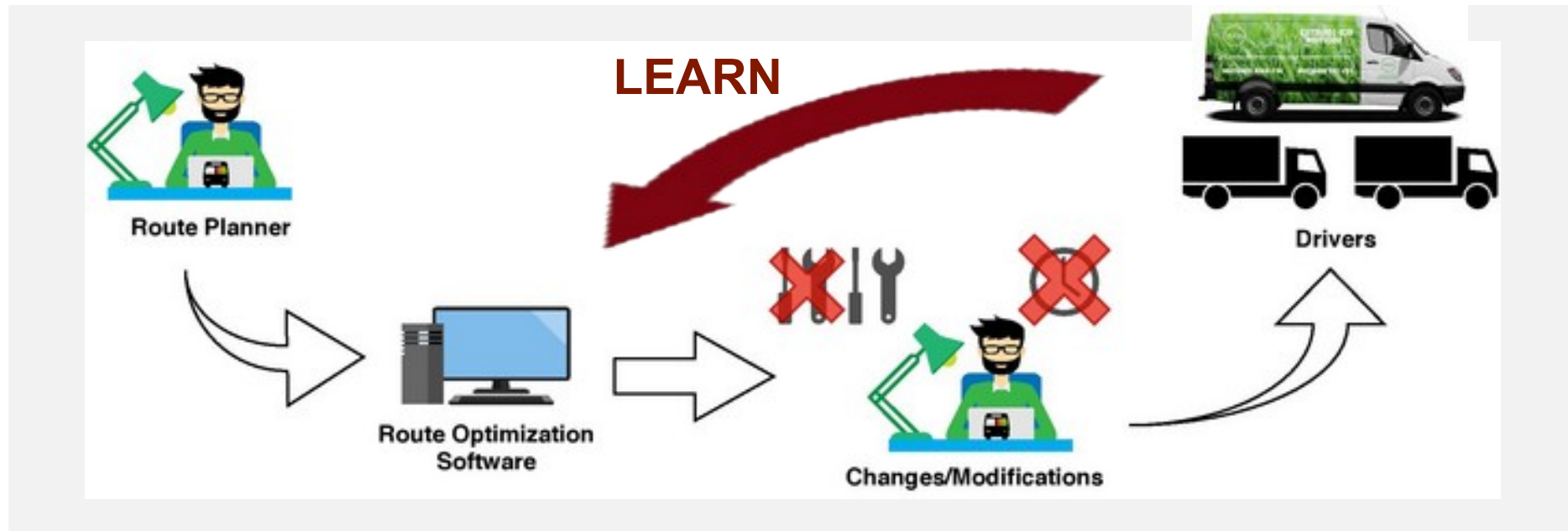
“in real-life operations, the quality of a route is not exclusively defined by its theoretical length, duration, or cost”

- Data:
 - stop list, zones
 - TSP solutions with “good, average, bad” labels
- learn and route



Preferences from historic solutions

Learn-n-route:



GOAL: Learn preferences, reduce manual effort, adapt to changes over time!

Tacit knowledge (user preferences)

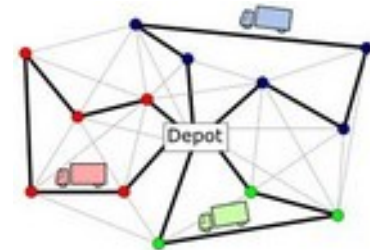
For single vehicles, in mobility data mining literature:

- ▶ Driver turn prediction [Krumm, 2008]
- ▶ Prediction of remainder of route early in the trip [Ye et al., 2015]
- ▶ Prediction of route given origin and destination [Wang et al., 2015]



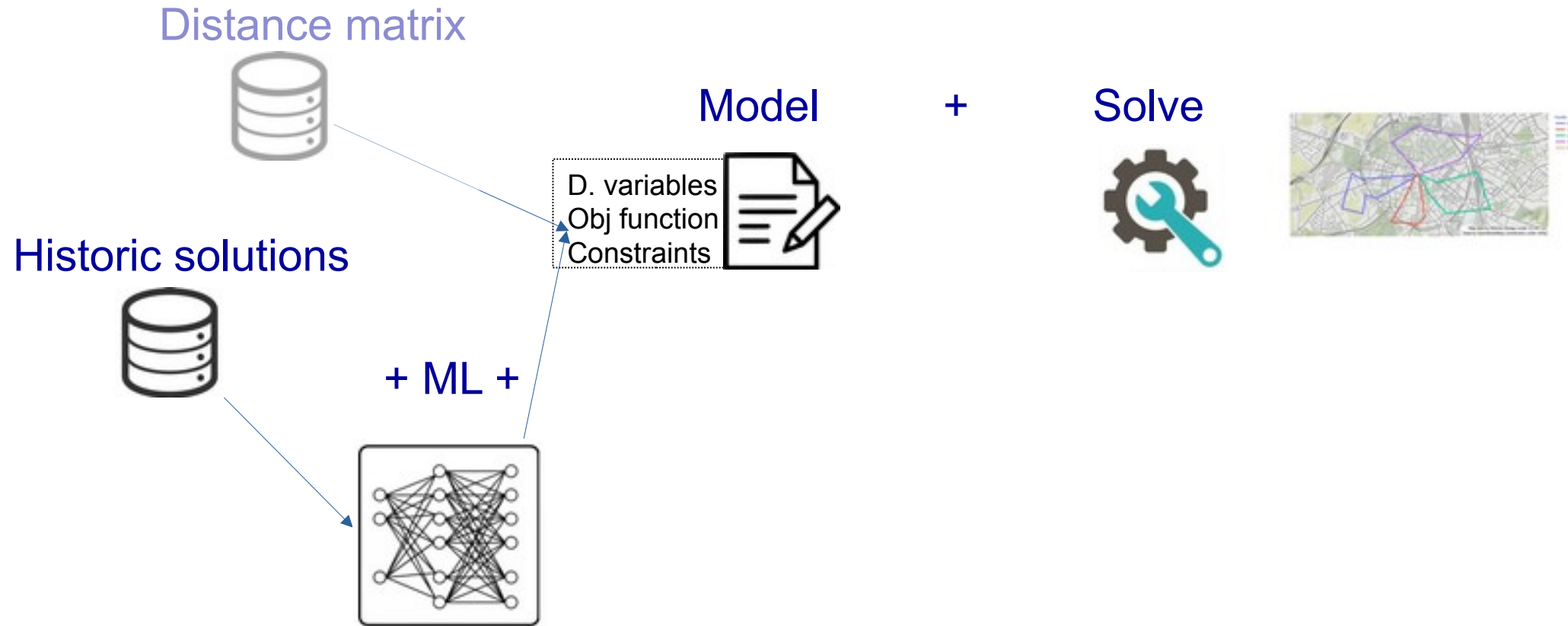
Can we use similar techniques
to learn preferences across routings of multiple vehicles?

And can we optimize over them with constraint solving?



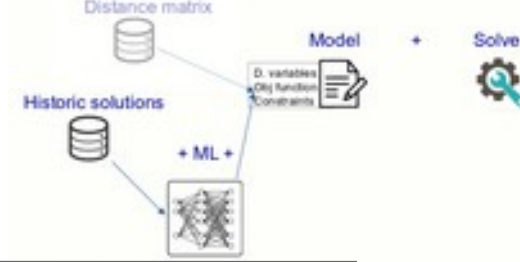
Preferences from historic solutions

Learn-n-route:



[Learn-n-Route: Learning implicit preferences for vehicle routing. Canoy et al, CONS23]

Learning and prediction part



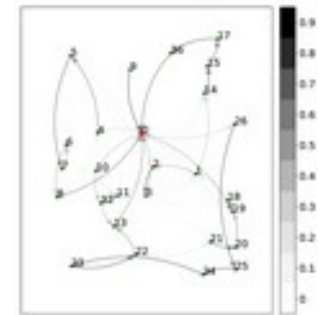
1st order Markov approximation:

$$P([s_1, s_2, s_3, \dots]) = P(s_1) * P(s_2 | s_1) * P(s_3 | s_2) * \dots$$

→ estimate the $P(s_y | s_x)$ by observing the transitions in the actually driven routes (*subset of known clients each day*)

probability of transition = relative nr of observations in the data

$$t_{ij} = \frac{f_{ij} + \alpha}{N_i + \alpha d},$$



[Canoy, Guns, Vehicle routing by learning from historical solutions, CP19]

Preference-based: joint inference

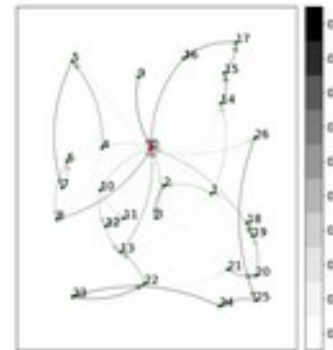
- First-order Markov approximation of the tour preference:

$$P(\langle \text{depot}, s_1, s_2, \dots \rangle) \sim P(\text{depot}) * P(s_1 | \text{depot}) * P(s_2 | s_1) * \dots$$

- Joint-inference over transition-probability matrix:

$$\max \prod_{(i,j) \in X} \Pr(\text{next stop} = j \mid \text{current stop} = i),$$

$$\max \sum_{(i,j) \in A} \log(t_{ij}) x_{ij} \quad \text{s.t. VRP}(x_{ij})$$



= standard VRP formulation

(2nd order: quadratic objective)

(3rd order: cubic objective...)

Compatible with ALL vrp solvers

[Learn-n-Route: Learning implicit preferences for vehicle routing. Canoy et al, CONS23]

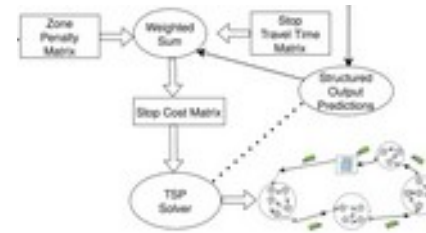
Preference-based: learning?

Given: historic solutions (positive-only)

1) Non-contextual:

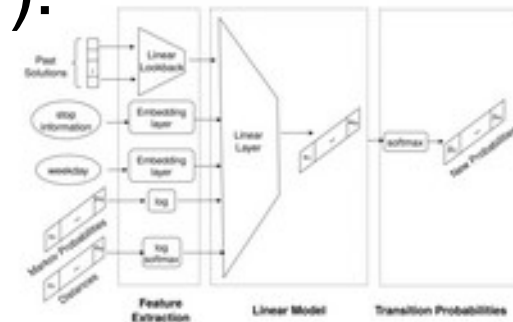
- Simple histogram counting
- 2-stage Structured Perceptron

$$t_{ij} = \frac{f_{ij} + \alpha}{N_i + \alpha d},$$



2) Contextual (day of week, other stops, ...):

- Neural network, max likelihood
- Data Driven Inverse Opt (e2e with solving)
does NOT scale!



[Learn-n-Route: Learning implicit preferences for vehicle routing. Canoy et al, CONS23]
[Probability estimation and structured output prediction for learning preferences in last mile delivery. Canoy et al]

Preference-based: reweighing for concept drift

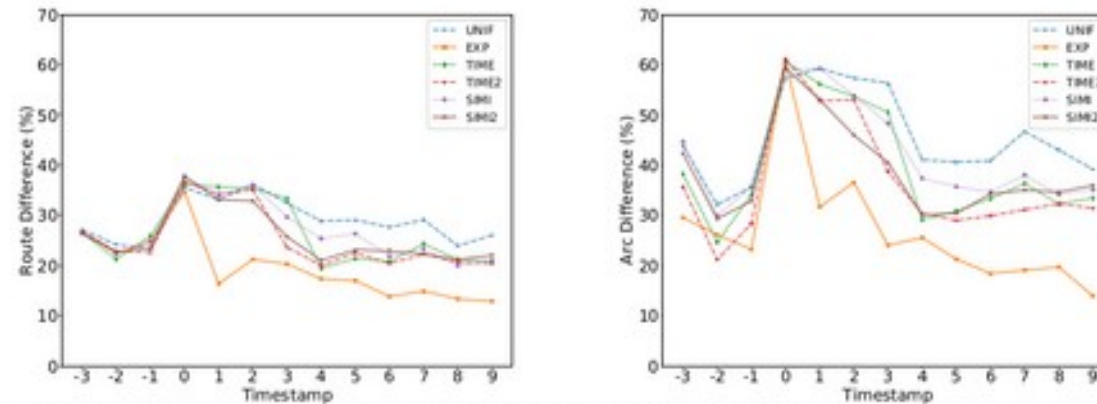


Fig. 7 Route and arc difference during concept drift (drop in number of stops)

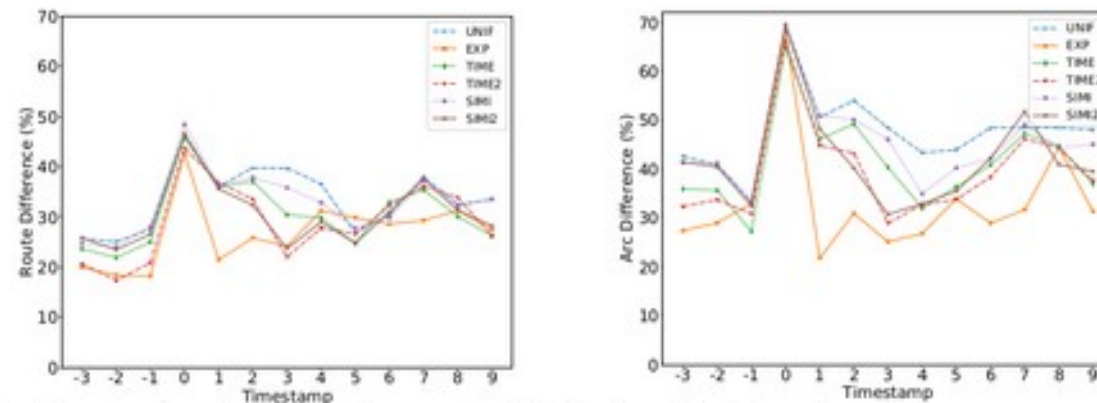
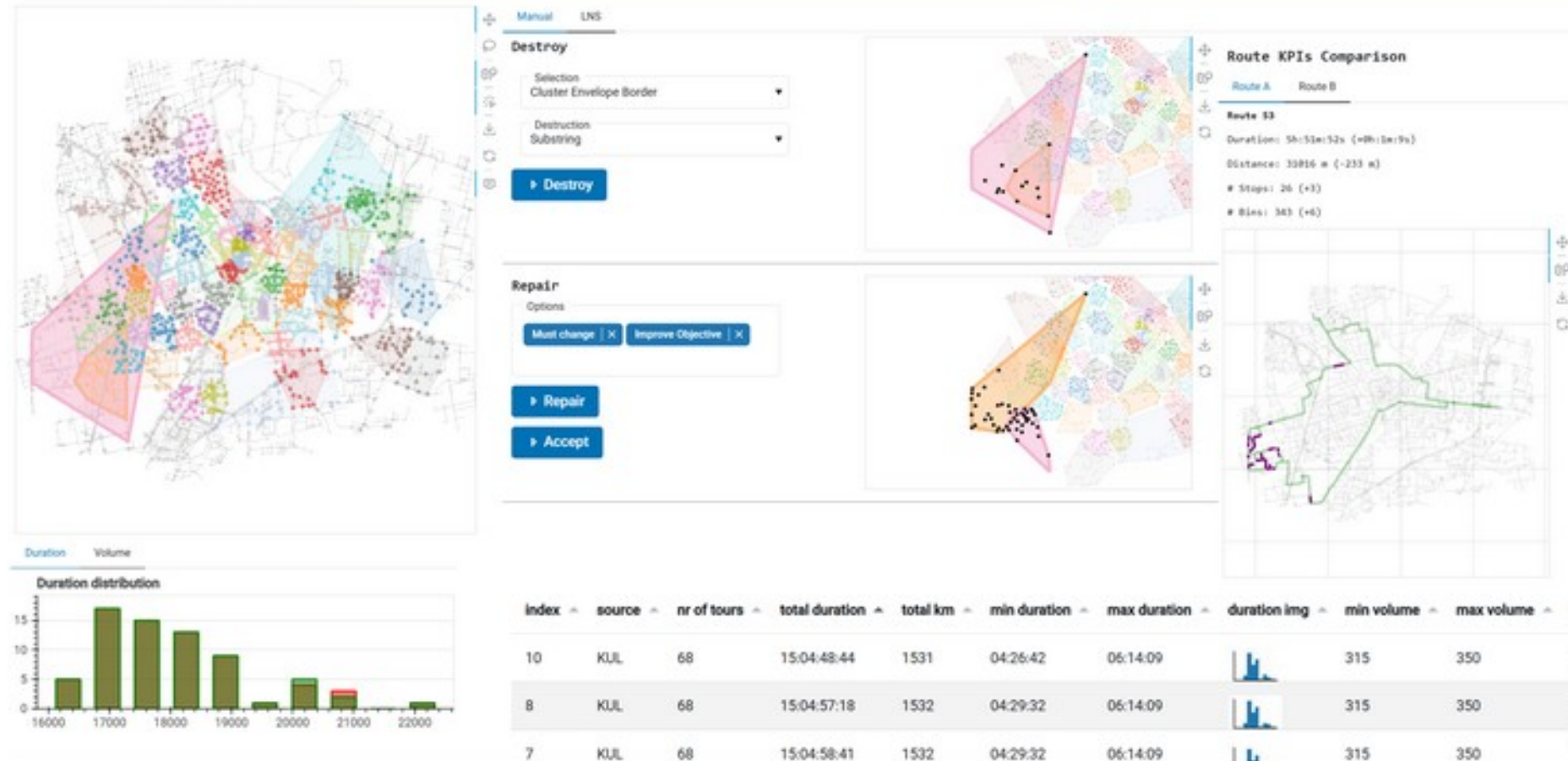


Fig. 8 Route and arc difference during concept drift (rise in number of stops)

[Learn-n-Route: Learning implicit preferences for vehicle routing. Canoy et al, CONS23]

Multiple-objective Optimisation

TUPLES - Waste-Collection case



Active preference learning for Comb Opt

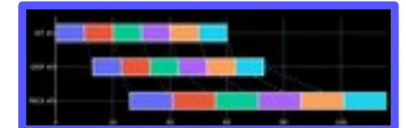
- Multi-objective optimisation (n objs.)

$$\hat{y} = \operatorname{argmax}_{y \in \mathcal{F}} u(y) = \langle w, \phi(y) \rangle = \sum_{i=1}^n w_i \phi_i(y)$$

- Constructive Preference Elicitation [Dragone et al, 2018]

Algorithm 1 Template for CPE

```
Initial weight estimate  $w^0$ 
for  $t < T$  do
  Observe problem instance  $p^t$ 
   $(y_1, y_2) \leftarrow \text{SELECT QUERY}(p^t, w^t)$ 
  DM chooses  $y_+$  from  $(y_1, y_2)$ 
   $w^{t+1} \leftarrow \text{WEIGHT UPDATE}(w^t, y_1, y_2)$ 
end for
return  $\hat{y} = \operatorname{argmax}_{y \in \mathcal{F}} \langle w^T, \phi(y) \rangle$ 
```



or?



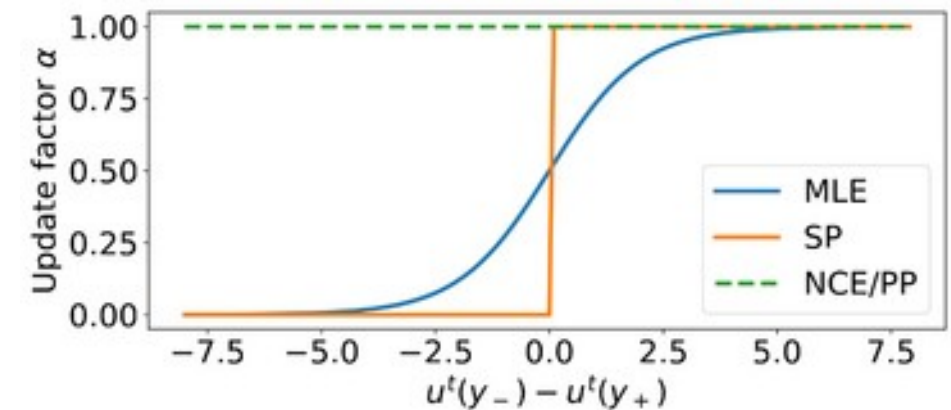
Active preference learning for Comb Opt $= \sum_{i=1}^n w_i \phi_i(y)$

Weight Update (user chose y_+)

$$w^{t+1} = w^t + \eta \cdot \alpha^t \Delta \quad \Delta = \phi(y_+, p) - \phi(y_-, p)$$

Unified view:

- Pref. Perceptron/InfoNCE: $\alpha^t = 1$
- Struct. Perceptron: $\alpha^t = \mathbb{1}_{u^t(y_+) < u^t(y_-)}$
- Max. Likelihood of Bradley-Terry pref:
 $\alpha^t = \mathcal{S}(u^t(y_-) - u^t(y_+))$



Active preference learning for Comb Opt



or?



Select Query = solve two comb opt problems:

$$y_1 = \operatorname{argmax}_{y \in \mathcal{F}} u(y) = \sum_{i=1}^n w_i \phi_i(y)$$

$$y_2 = \operatorname{argmax}_{y \in \mathcal{F}} (1 - \gamma)u(y) + \gamma \|\phi(y_1) - \phi(y)\|_1 \quad \text{diversity wrt. } y_1$$

- 2 x NP-hard + restricted to linear diversity/exploration functions

Our approach:

- Pre-computed solution pool
- UCB-based acquisition function for y_2 (non-linear)
- Cluster solutions for additional diversity (requires pool)

Active preference learning for Comb Opt

Query	Configuration						Prize-Collecting TSP					
	ChoicePerc			UCB, rpool (ours)			ChoicePerc			UCB, rpool (ours)		
	Regret (%)	#Q	% DM	Regret (%)	#Q	% DM	Regret (%)	#Q	% DM	Regret (%)	#Q	% DM
PP-online	6.6 ± 0.5	51	55	7.7 ± 0.5	67	45	2.7 ± 0.2	66	75	1.1 ± 0.05	29	100
SP-online	14.6 ± 0.7	-	20	8.9 ± 0.5	69	55	12.1 ± 0.7	-	40	1.2 ± 0.06	22	100
MLE-online	8.2 ± 0.6	90	65	4.9 ± 0.6	70	60	1.7 ± 0.1	39	90	0.6 ± 0.03	24	100
MLE-batch	8.6 ± 0.6	47	50	2.2 ± 0.2	50	80	1.4 ± 0.1	25	100	0.5 ± 0.02	20	100

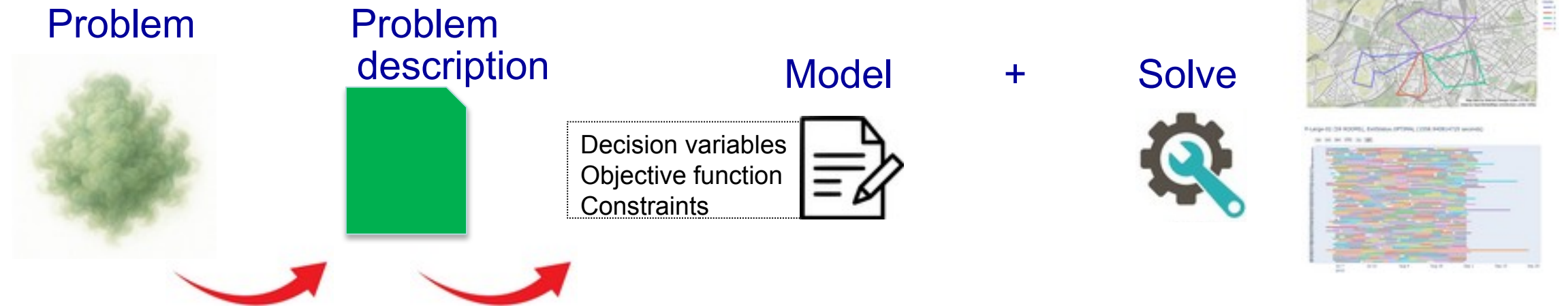
[Preference Elicitation for Multi-objective Combinatorial Optimiza... Defresne et al. IJCAI-25]

Bigger picture

- Learning from images, production data, ...
- Learning user preferences
- **Stateful interaction**



Model + Solve paradigm, in practice



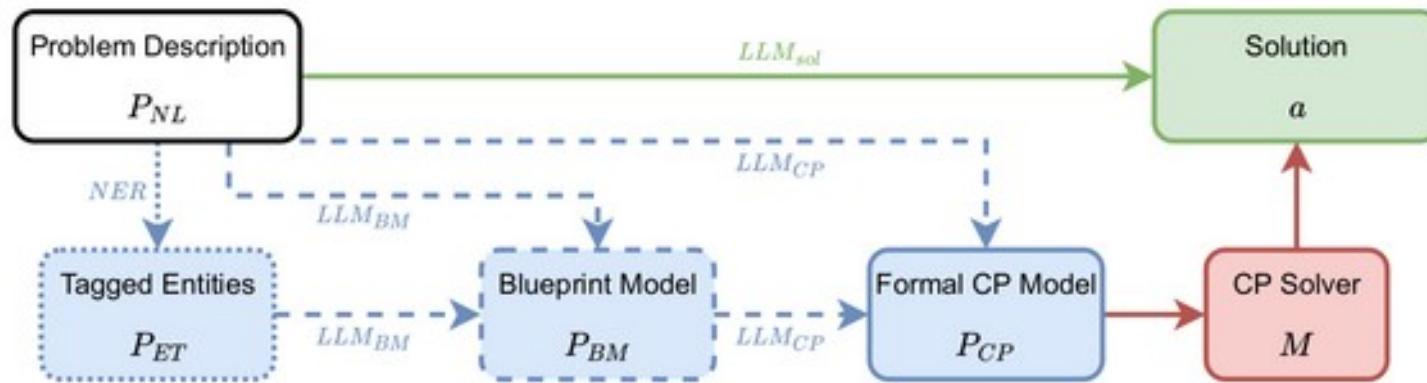
Who translates problem description into a formal model?

A: it requires expertise... so an expert

Q: can we use AI to support formal modeling?

Using LLMs to help solve CO problems?

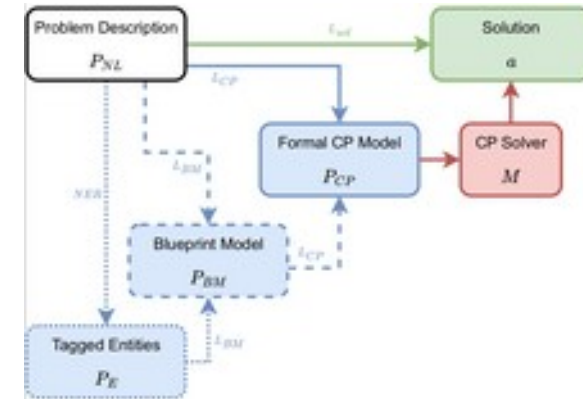
A retired professor wants to invest up to \$50000 in the airline and railway industries. Each dollar invested in the airline industry yields a \$0.30 profit and each dollar invested in the railway industry yields a \$0.10 profit. A minimum of \$10000 must be invested in the railway industry and at least 25% of all money invested must be in the airline industry. How to maximize the professor's profit?



[Michailidis K, Tsouros D, Guns T. Constraint modelling with LLMs using in-context learning. CP 2024]

Evaluation

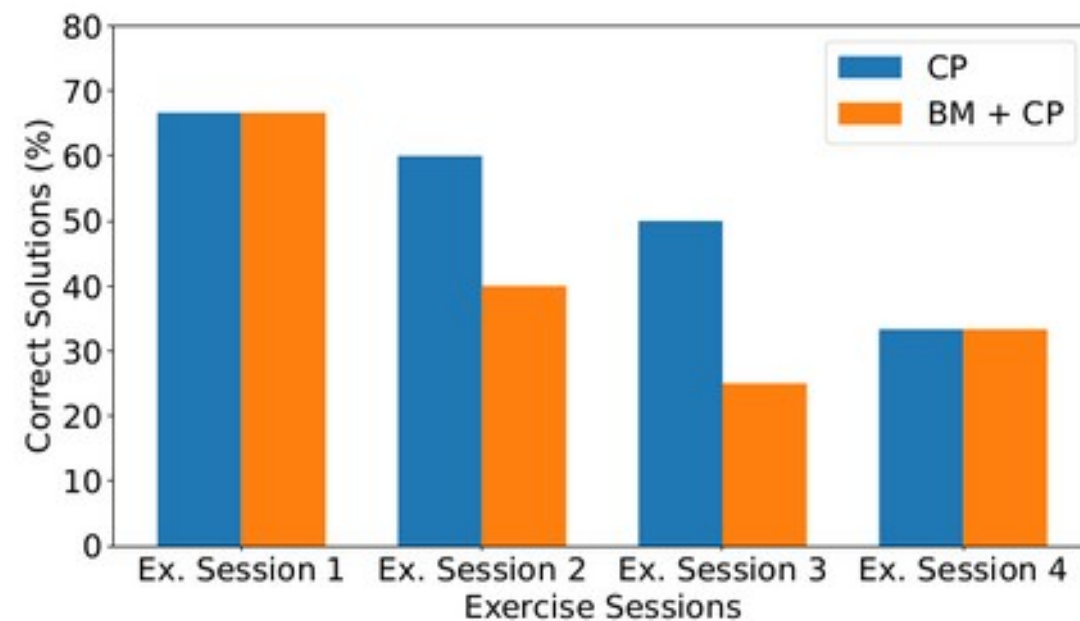
- ❑ we evaluate **in-context learning** approaches in a **multi-stage** pipeline
- ❑ **Automatic evaluation** can be tricky!
 - ~ To check an assignment, decision variable names must match!
 - ~ Have to do name-based similarity matching with ground truth, and/or prompt guiding on the variable names
- ❑ What to **measure**?
 - ~ Solution accuracy (LGP, but may not be unique)
 - ~ Declaration accuracy (NL4Opt, but syntactic variations)
 - ~ Model accuracy (new, check logical equivalence of entire model)



[Michailidis K, Tsouros D, Guns T. Constraint modelling with LLMs using in-context learning. CP 2024]

Dataset	Method	#Err	Accuracy (%)		
			Solution	Declaration	Model
NL4Opt	Direct	1	11.46	N/A	N/A
	CP	7	81.31	87.81	79.24
	+ BM	8	84.43	89.93	82.01
	+ NER	8	85.47	88.60	80.62
LGPs	Direct	4	9.36	N/A	N/A
	CP	11	57.00	80.45	55.00
	+ BM	18	58.00	70.69	58.00
	+ NER	20	54.00	67.77	50.00

Constraint Solving course exercises



Better evaluation, and better integration

- Need for curated, diverse dataset of constraint programming problems...

New: CP-Bench

<https://huggingface.co/datasets/kostis-init/CP-Bench>

Leaderboard (open source eval scripts):

<https://huggingface.co/spaces/kostis-init/CP-Bench-Leaderboard>

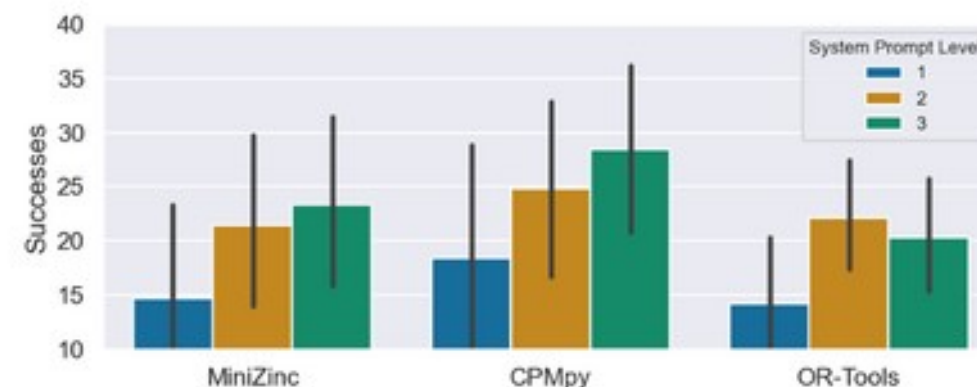
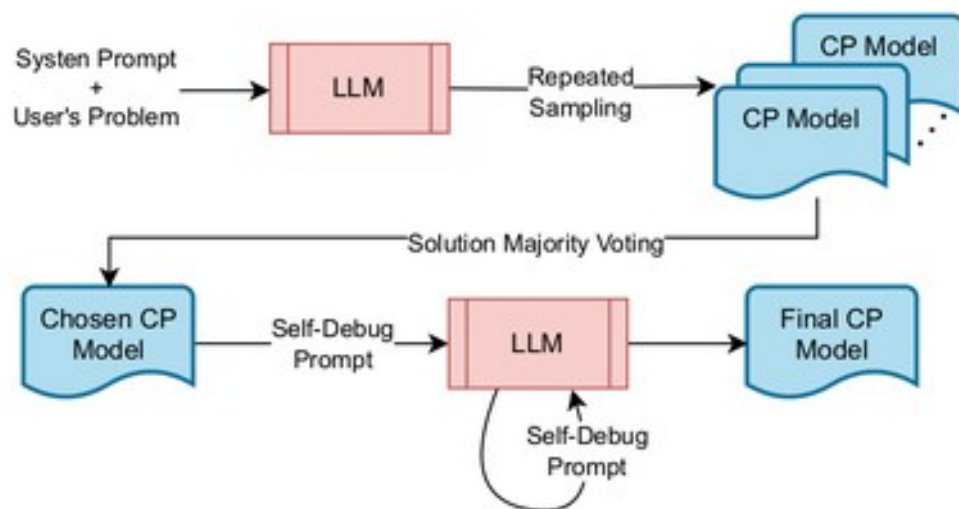
Results Leaderboard

Name	Modelling Framewo...	Base LLM	Models Submitted (%)	Accuracy (...)	Runtime Error
multi_agents_requirements_ga...	OR-Tools	o3	100	71.29	0.99
sampling_10_and_self_debug_10	CPMpy	gpt-4.1-mi	100	69.31	1.98

aplai_course_1_bank_card
aplai_course_1_five_floors
aplai_course_1_guards_and_ap
aplai_course_1_magic_square
aplai_course_1_money_change
aplai_course_1_thick_as_thieve
aplai_course_2_color_simple
aplai_course_2_maximal_indep
aplai_course_2_movie_scheduli
aplai_course_2_subsets_100
aplai_course_2_subset_sum
aplai_course_3_farmer_and_co
aplai_course_3_kidney_exchang
aplai_course_3_people_in_a_ro
aplai_course_5_climbing_stairs
aplai_course_5_grocery
aplai_course_5_hardy_1729_sq
cpmpy_examples_bus_scheduli
cpmpy_examples_jobshop
cpmpy_examples_minesweeper
cpmpy_examples_packing_rect
cpmpy_examples_resource_cor
cpmpy_examples_room_assign
cpmpy_examples_send_more_r
cpmpy_examples_set_game
cpmpy_examples_sudoku
cpmpy_examples_who_killed_a
cpmpy_examples_wolf_goat_ca
cpmpy_examples_zebra
csplib_csplib_001_car_sequenc
csplib_csplib_005_autocorrelati
csplib_csplib_008_vessel_loadin
csplib_csplib_009_perfect_squa
csplib_csplib_012_nonogram
csplib_csplib_015_schurs_lemm
csplib_csplib_021_crossfigures
csplib_csplib_053_graceful_grap
csplib_csplib_084_hadamard_m
hakan_examples_abbots_puzzl
hakan_examples_added_corner
hakan_examples_ages_of_the
hakan_examples_allergy
hakan_examples_appointment
hakan_examples_archery_puzz
hakan_examples_assignment_c
hakan_examples_autoref
hakan_examples_bin_packing
hakan_examples_cabling
hakan_examples_candies
hakan_examples_capital_budg
hakan_examples_chess_set
hakan_examples_circling_squar
hakan_examples_coin3_applica
hakan_examples_coins_grid
hakan_examples_contracting_c
hakan_examples_covering_opl
hakan_examples_crossword
hakan_examples_crypta
hakan_examples_eighteen_hole
hakan_examples_facility_locati
hakan_examples_fifty_puzzle
hakan_examples_three_sum
hakan_examples_twelve_pack

Better evaluation, and better integration

- More and diverse combinatorial optimisation descriptions (~100 now)
- 'tool calling' setup: LLM calls solver and interprets output (errors and solutions)



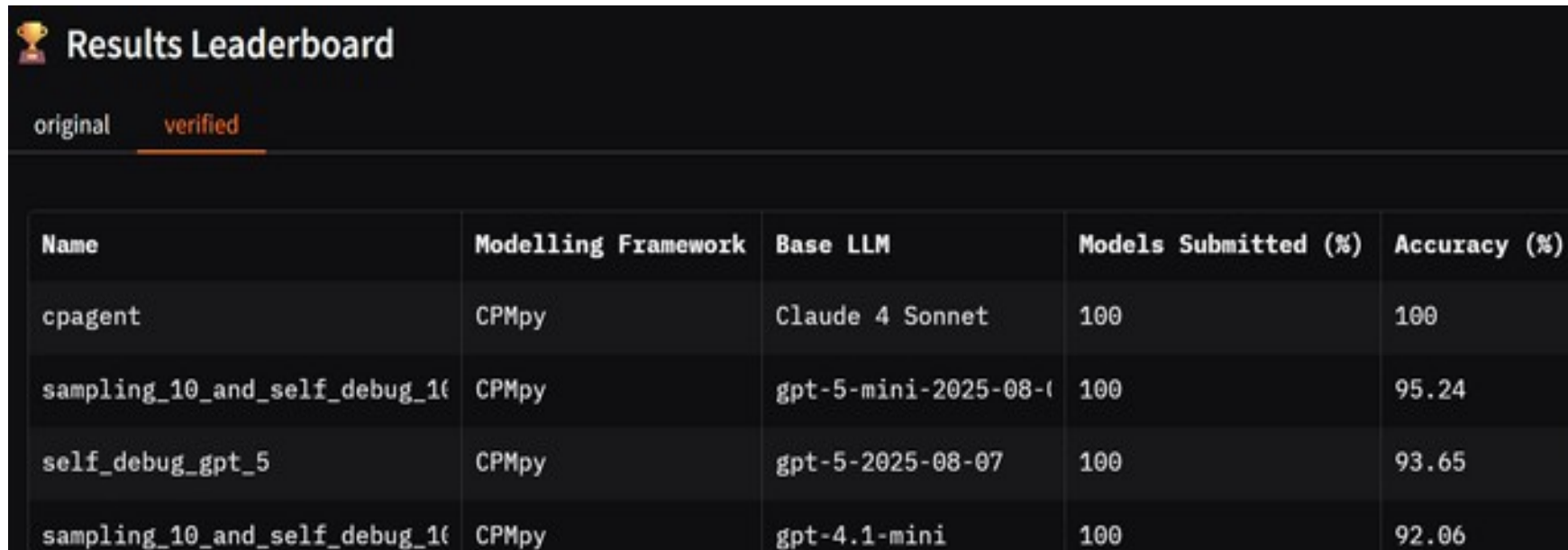
High-level specification

Python

[Michailidis K, Tsouros D, Guns T. Ongoing work...]

LLM-based modeling?

- CP-Bench (60+ models): <https://huggingface.co/datasets/kostis-init/CP-Bench>
- HF Leaderboard: <https://huggingface.co/spaces/kostis-init/CP-Bench-Leaderboard>



 **Results Leaderboard**

original verified

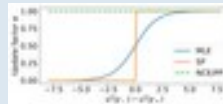
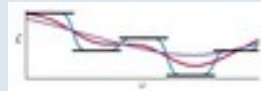
Name	Modelling Framework	Base LLM	Models Submitted (%)	Accuracy (%)
cpagent	CPMpy	Claude 4 Sonnet	100	100
sampling_10_and_self_debug_16	CPMpy	gpt-5-mini-2025-08-07	100	95.24
self_debug_gpt_5	CPMpy	gpt-5-2025-08-07	100	93.65
sampling_10_and_self_debug_16	CPMpy	gpt-4.1-mini	100	92.06

Also: **live** chat-opt demo: <https://chatopt.cs.kuleuven.be>

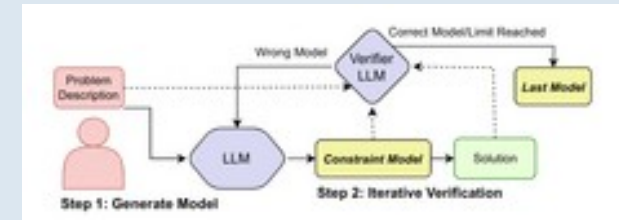
[Michailidis K, Tsouros D, Guns T. ECAI 25]

Learning: underspecified CO problems

- Underspecified parameters:
 - **From (image) classifiers***
 - From (contextual) regression*
- Underspecified preferences:
 - Learn from historic solutions
 - Learn from interaction with user*
- Underspecified structure:
 - Interactive Constraint Acquisition



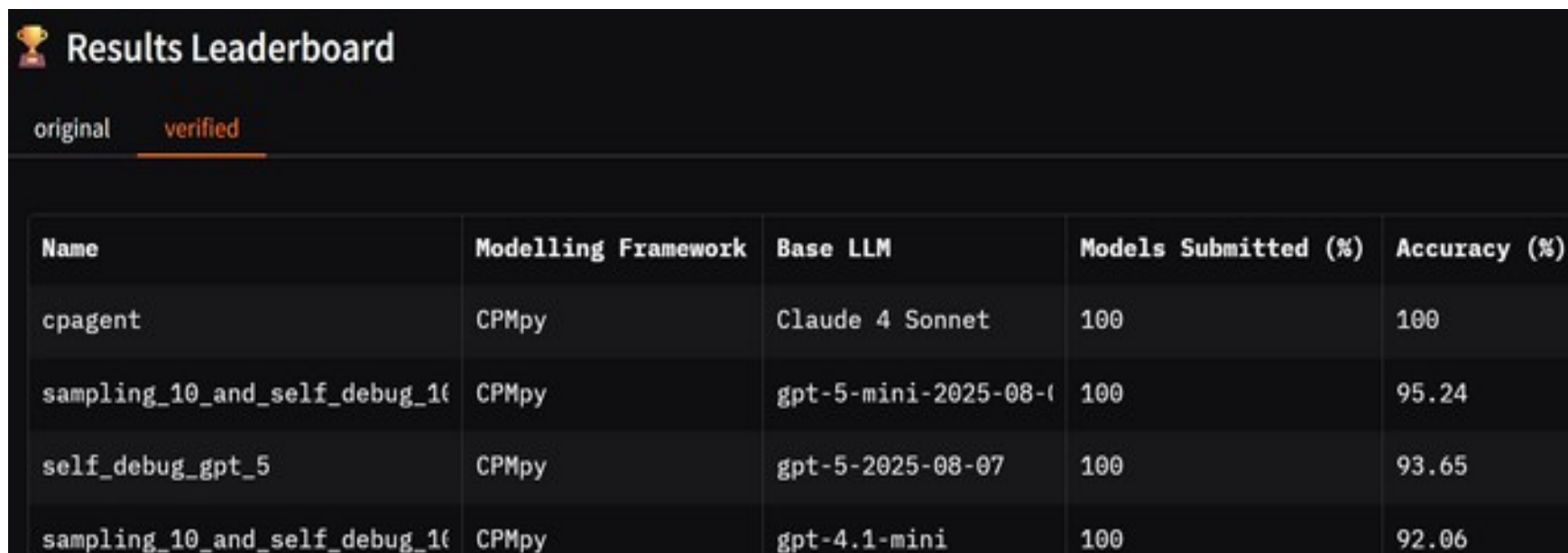
Conversational



Run	Modeling Framework	Base LLM	Models Submitted (N)	Accuracy (N)
original	CPay	Claude 4 Sonnet	100	100
sampling_10_and_self_debug_10	CPay	gpt-5-mini-2025-08-1	100	95.34
self_debug_gpt_5	CPay	gpt-5-2025-08-07	100	93.45
sampling_10_and_self_debug_10	CPay	gpt-4.1-mini	100	92.86

LLM-based modeling?

- CP-Bench (60+ models): <https://huggingface.co/datasets/kostis-init/CP-Bench>
- HF Leaderboard: <https://huggingface.co/spaces/kostis-init/CP-Bench-Leaderboard>



 **Results Leaderboard**

original verified

Name	Modelling Framework	Base LLM	Models Submitted (%)	Accuracy (%)
cpagent	CPMpy	Claude 4 Sonnet	100	100
sampling_10_and_self_debug_16	CPMpy	gpt-5-mini-2025-08-06	100	95.24
self_debug_gpt_5	CPMpy	gpt-5-2025-08-07	100	93.65
sampling_10_and_self_debug_16	CPMpy	gpt-4.1-mini	100	92.06

Also: **live** chat-opt demo: <https://chatopt.cs.kuleuven.be>

[Michailidis K, Tsouros D, Guns T. ECAI 25]

Leren en redeneren

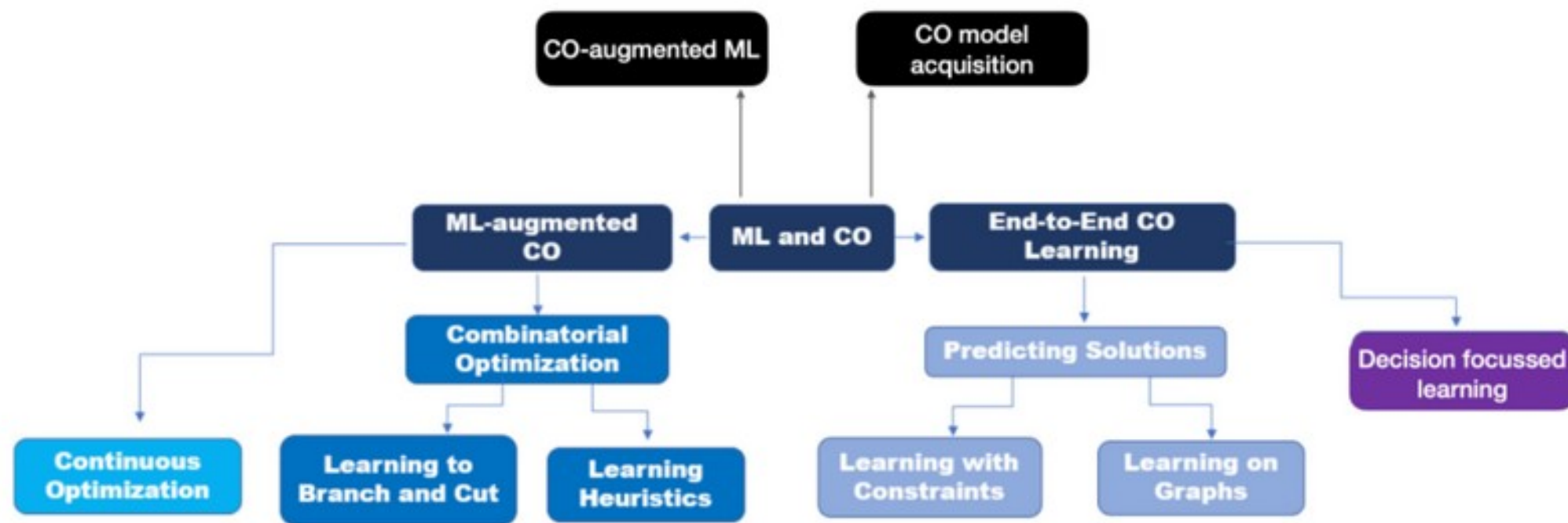
Henry Kautz' AAAI2020 categorisatie*:

		Voorbeelden deze les?
[symbolic Neuro symbolic]	Input and output symbolic, all processing is neural	
[Neuro[Symbolic]]	Symbolic reasoning is activated by a neural engine	
[Symbolic[Neuro]]	Symbolic solver has a neural prediction component	
[Neuro + compile(Symbolic)]	Symbolic rules compiled away during training	
[Neuro → Symbolic]	Neural system provides input to symbolic system	

*Gebaseerd op bespreking in [Sarker, Zhou, Everhart, Hitzler. AI Comm 2021]

Neuro-Symbolic optimisation

Slide courtesy of N. Yorke-Smith

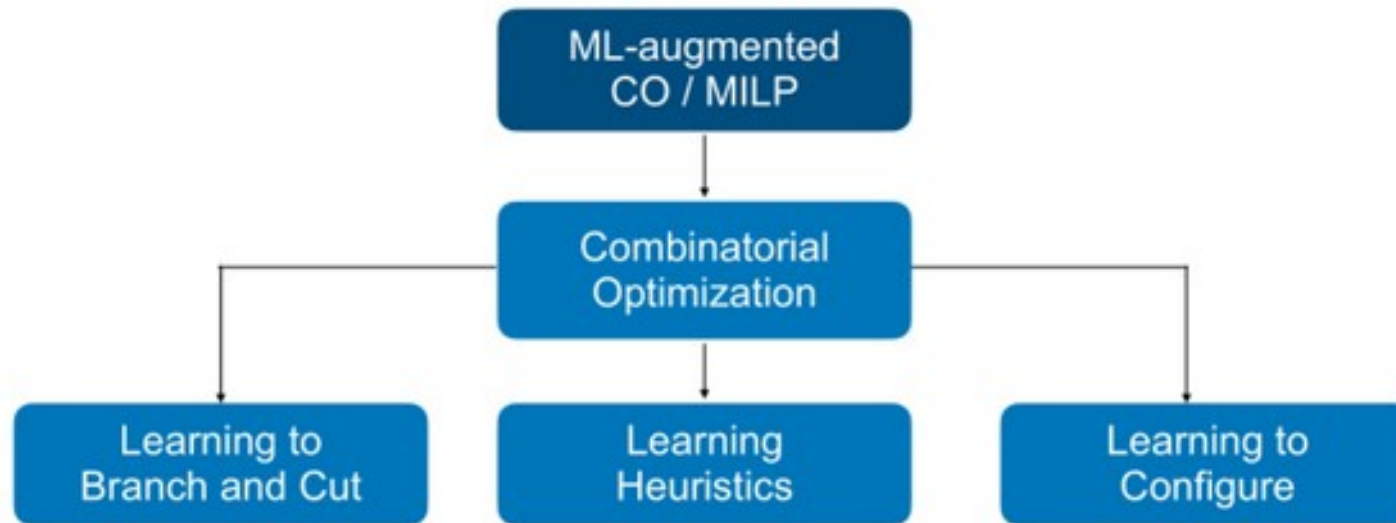


Y. Bengio, A. Lodi, A. Prouvost: [Machine Learning for Combinatorial Optimization: a Methodological Tour d'Horizon](#), EJOR 2021, 405-421

J. Kotary, F. Fioretto, P. Van Hentenryck, B. Wilder: [End-to-End Constrained Optimization Learning: A Survey](#). IJCAI 2021: 4475-4482

[Symbolic[Neuro]]

ML-augmented MILP



L. Scavuzzo, K. Aardal, A. Lodi, N. Yorke-Smith: **Machine Learning Augmented Branch and Bound for Mixed Integer Linear Programming**, arXiv:2402.05501, 2024, Mathematical Programming <https://doi.org/10.1007/s10107-024-02130-y>

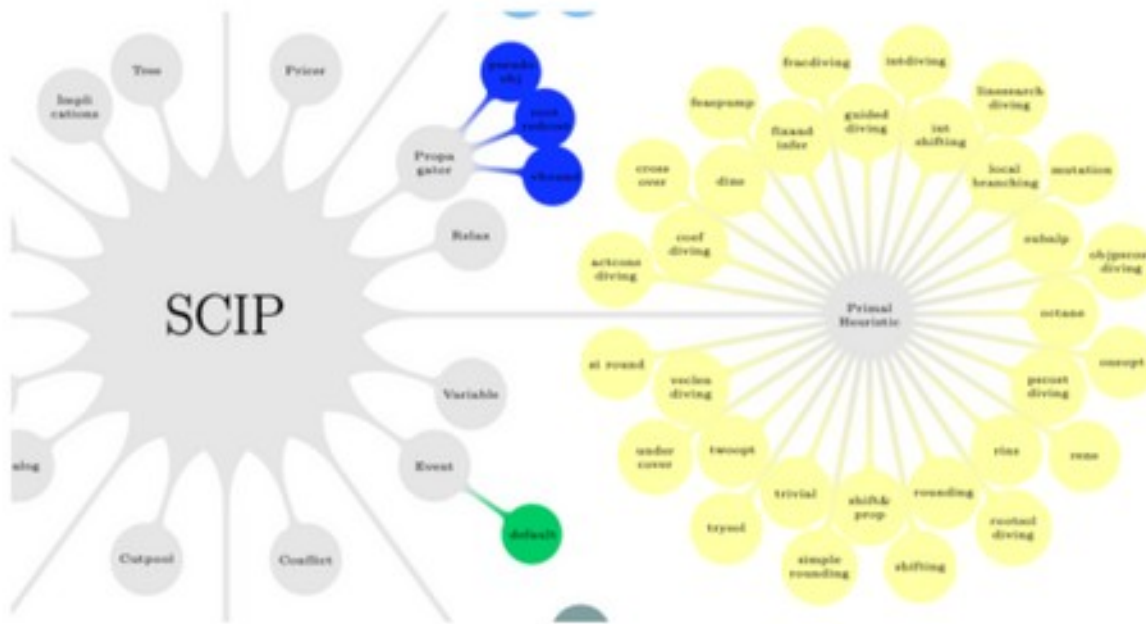
[Symbolic[Neuro]]

Exact Combinatorial Optimization with Graph Convolutional
Neural Networks [Gasse et al, NeurIPS19]

(separate slide deck)

[Symbolic[Neuro]]

Other learning task: Primal Heuristics



Once the MILP solvers have started to be reliable, practitioners recognized the need of producing good feasible solutions early in the process.

Primal heuristics are those algorithms that are run during B&B to either producing feasible solutions or improving them.

[Symbolic[Neuro]] for primal heuristics

A **number of methodologies** have been proposed for this purpose.

Conceptually, they can be split into **three main approaches**:

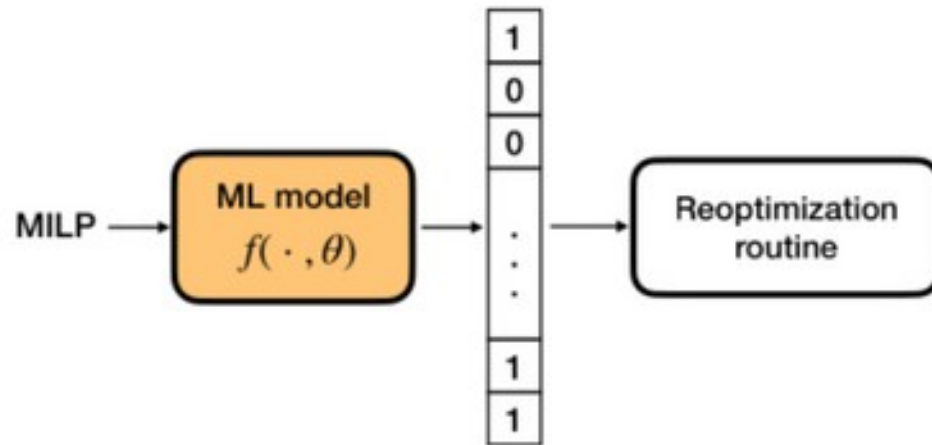
- (a) *guiding a heuristic search* with a starting predicted solution,
- (b) *solution improvement* via a learned neighborhood selection criterion, and
- (c) *learning a schedule* to pre-existing heuristic routines.

For (a) and (b), the important concept is that of **Large Neighborhood Search**.

The idea is to optimize an **auxiliary MILP of smaller size**, constructed by **reducing the feasible region** of the original MILP.

Typically done by **fixing the value of some of the variables** and optimizing the rest.

[Symbolic[Neuro]] for primal heuristics

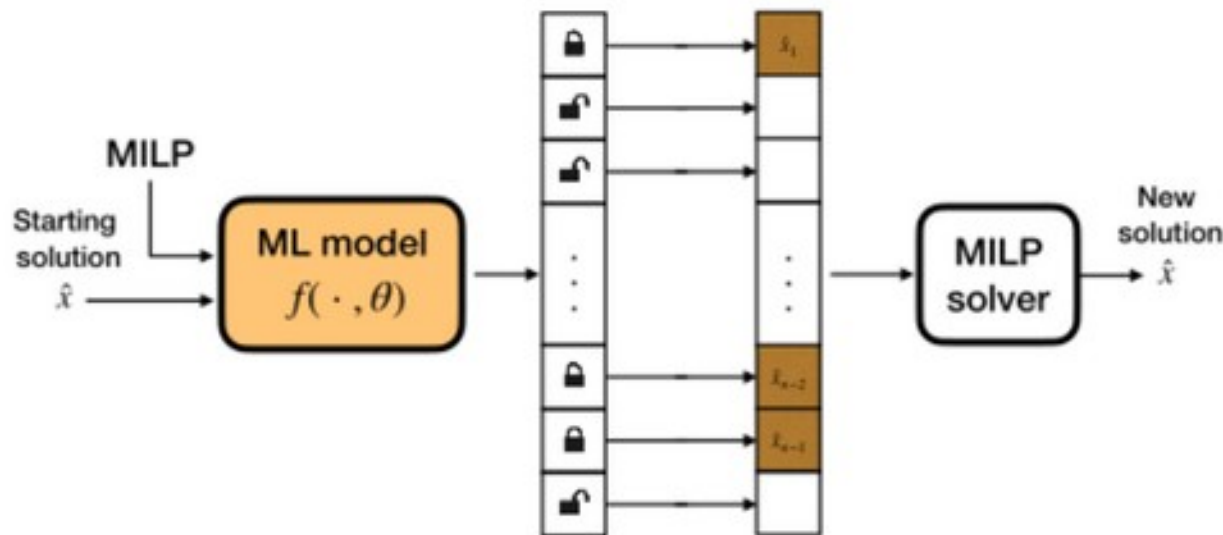


Guiding a heuristic search with a starting predicted solution:

- The goal is to produce a (partial) assignment of the binary variables in a binary or mixed binary MILP, that can then be used to guide the search.
- Often, this is obtained by starting from a set of collected solutions.

Examples in Ding et al. (2020), Nair et al. (2020) and Khalil et al. (2022).

[Symbolic[Neuro]] for primal heuristics



Neighborhood selection:

- **which** and/or
- **how many** variables to **unfix** and re-optimize.

The goal is to identify **substructures** of the problem that can be used **to decompose it** into smaller, more manageable sub-problems.

Examples in Song et al. (2020), Wu et al. (2021) and Liu et al. (2022).

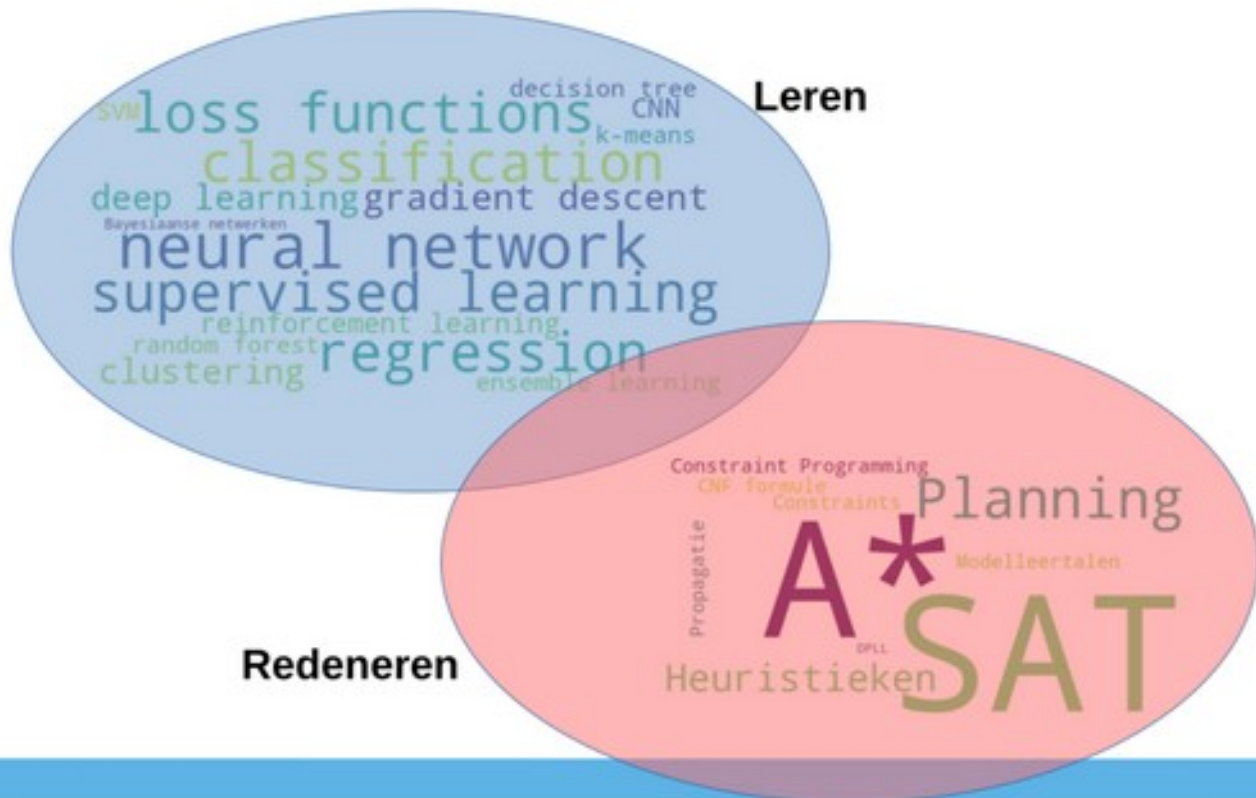
[Symbolic[Neuro]] for MIP

Challenges and opportunities remain:

- Challenge: The use of NNs and especially GNNs has proven effective for the considered learning tasks. However, computation on NNs is especially effective by using **GPUs**, while classical MILP computation is on **CPUs**. (more recently: same with *transformers*)
- Opportunity: The use of restart mechanisms and re-optimisation in MILP is increasing and the chance of using the **information collected by running the solvers for a limited amount of time** seems to favor the use of ML even more.

Conclusie: redeneertechnieken deze les

- Optimalisatie
(CP, MIP, MaxSAT etc)



Conclusie: leren en redeneren

Henry Kautz' AAAI2020 categorisatie*:

		Voorbeelden deze les
[symbolic Neuro symbolic]	Input and output symbolic, all processing is neural	...
[Neuro[Symbolic]]	Symbolic reasoning is activated by a neural engine	
[Symbolic[Neuro]]	Symbolic solver has a neural prediction component	
[Neuro + compile(Symbolic)]	Symbolic rules compiled away during training	
[Neuro → Symbolic]	Neural system provides input to symbolic system	

*Gebaseerd op bespreking in [Sarker, Zhou, Everhart, Hitzler. AI Comm 2021]

Einde

